ALMA MATER STUDIORUM UNIVERSITY OF BOLOGNA
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING – DISI

Transformers Under The Hood

''Why did the seq2seq break up with RNN? It found someone who gave it better attention''

Prof. Gianluca Moro
Tutors: Dr. Giacomo Frisoni, Dr. Lorenzo Molfetta

DISI, University of Bologna, Cesena
Via dell'Università, 50 I-47522 Cesena (FC), Italy
{name.surname}@unibo.it

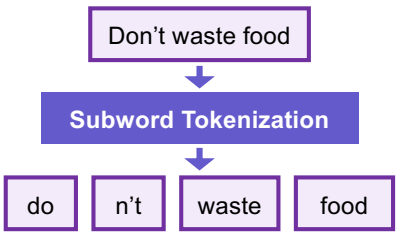
Outline

- Tokenization
- Seq2seq
- Transformer
- Decoding Strategies

Tokenization: A Necessary Premise

Tokens: How Neural Networks «See»

- Neural networks cannot work directly on text data
  - Text must be converted into numerical representations
- Tokenization is a crucial step in todays' NLP
  - The process of breaking down a piece of text into **smaller indivisible units**, called **tokens**
  - Deep Learning architectures in NLP process the raw text at the token level
  - Tokens can be **words**, **subwords** (n-gram characters), or **characters**



Word	Token(s)
surf	['surf']
surfing	['surf', '##ing']
surfboarding	['surf', '##board', '##ing']
surfboard	['surf', '##board']
snowboard	['snow', '##board']
snowboarding	['snow', '##board', '##ing']
snowing	['snow', '##ing']

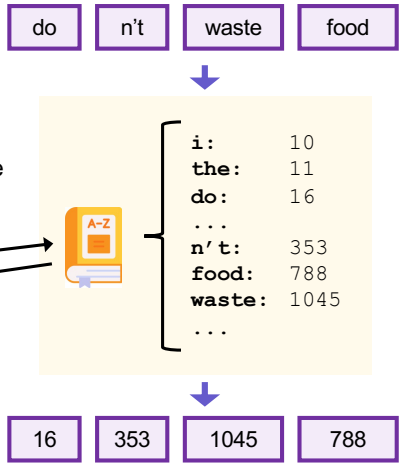
Tokenization is deeper than it seems

- The network can better understand the semantic similarities between surf**board**ing and snow**board**ing, **snow**ing and **snow**boarding, etc.

i tokenizer non usano reti neurali né l'apprendimento basato sul gradiente (backpropagation) per decidere come dividere il testo, ma si basano su algoritmi deterministici e conteggi statistici (Il tokenizer non ha pesi addestrabili: i tokenizer moderni vengono costruiti prima dell'addestramento del modello analizzando un vasto corpus di testo tramite algoritmi euristici.).

Tokenization Algorithms and Vocabulary Mapping

- Tokenizers are not fully learned but based on linguistic algorithms
  - Whitespace tokenization
  - Punctuation-based tokenization
  - Regular expression tokenization
  - Entity-based tokenization (e.g., city names)
  - **Subword tokenization**
    - E.g., Byte Pair Encoding (BPE) iteratively merges the most frequent pairs of characters or subwords
  - Character tokenization



Each language model has a predefined vocabulary of tokens on which it is trained on (e.g., GPT-2 vocabulary size = 50K, LLaMA-2=32K, LLaMA-3 = 128K)

⚠ *English has an estimated 600K words*

Different Models, Different Tokenizers

```
1 # Display the results
2 print("Original Text:", sample_text)
```

```
Original Text: Tokenizer architectures for LLMs are fascinating!
```

«##» indicates that the token is a continuation of the previous token

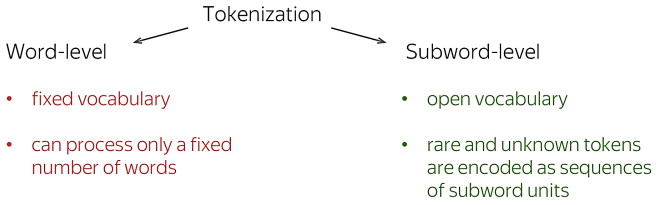
```
1 print("\nBERT (WordPiece) Tokens:", bert_tokens)
2 print("\nGPT-2 (Byte-Pair Encoding) Tokens:", gpt2_tokens)
3 print("\nRoBERTa (Byte-level BPE) Tokens:", roberta_tokens)
4 print("\nT5 (SentencePiece) Tokens:", t5_tokens)
```

```
BERT (WordPiece) Tokens: ['token', '##izer', 'architecture', '##s', 'for', 'll', '##ms', 'are', 'fascinating', '!']

GPT-2 (Byte-Pair Encoding) Tokens: ['Token', 'izer', 'Ġarchitectures', 'Ġfor', 'ĠLL', 'Ms', 'Ġare', 'Ġfascinating', '!']

RoBERTa (Byte-level BPE) Tokens: ['Token', 'izer', 'Ġarchitectures', 'Ġfor', 'ĠLL', 'Ms', 'Ġare', 'Ġfascinating', '!']

T5 (SentencePiece) Tokens: ['_To', 'ken', 'izer', '_architecture', 's', '_for', '_L', 'LM', 's', '_are', '_fascinating', '!']
```

«_» indicates a space

Word-Level vs Subword-Level Tokenization

- The fixed vocabulary problem
  - Input tokens not in the vocabulary are replaced with a special **UNK** («unknown») token
  - If we use a straightforward word-level tokenization (i.e., our tokens are words), we will be able to process a fixed number of words
  - We will get lots of unown tokens and our model will lose their meaning
- How can we represent all words, even those not seen in the training data?
  - If you are not familiar with a word, you are familiar with the parts it consists of: **subwords** (in the worst case, symbols)
  - Then, why do not we split the rare and unknown words into smaller parts?



Tokenizer: Not a Technical Detail – i

- The component that converts *text* to *tokens* is called **tokenizer**
  - It is trained separately from the language model
  - Essentially impossible to change afterwards without retraining
- It may seem like a technical detail but it is foundational to how LLMs operate

Andrej Karpathy
Co-founder of OpenAI

Tokenization is at the heart of much weirdness of LLMs. Do not brush it off.

- Why can't LLM spell words? **Tokenization**.
- Why can't LLM do super simple string processing tasks like reversing a string? **Tokenization**.
- Why is LLM worse at non-English languages (e.g. Japanese)? **Tokenization**.
- Why is LLM bad at simple arithmetic? **Tokenization**.
- Why did GPT-2 have more than necessary trouble coding in Python? **Tokenization**.
- Why did my LLM abruptly halt when it sees the string "<|endoftext|>"? **Tokenization**.
- What is this weird warning I get about a "trailing whitespace"? **Tokenization**.
- Why the LLM break if I ask it about "SolidGoldMagikarp"? **Tokenization**.
- Why should I prefer to use YAML over JSON with LLMs? **Tokenization**.
- Why is LLM not actually end-to-end language modeling? **Tokenization**.
- What is the real root of suffering? **Tokenization**.

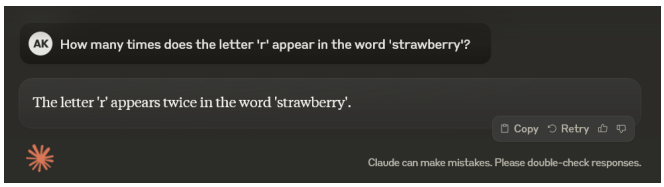
la tokenizzazione, pur venendo spesso percepita come un banale passaggio tecnico di pre-elaborazione del testo, determina in modo profondo i limiti, le capacità, i costi e persino i punti ciechi di un LLM.

Poiché il modello neurale non interagisce mai direttamente con le stringhe di testo ma lavora esclusivamente su sequenze di numeri (gli ID dei token), le scelte fatte dall'algoritmo di tokenizzazione influiscono direttamente sul comportamento dell'intelligenza artificiale nei seguenti modi:
- Incapacità di "vedere" le singole lettere: LLM non legge le lettere di una parola come fa un umano. Per la parola strawberry, il tokenizer potrebbe generare due token: [straw] e [berry]. Il modello non vede le singole lettere 'r', motivo per cui gli LLM spesso falliscono in compiti apparentemente banali come contare le lettere in una parola o invertire una stringa.
- Difficoltà nei calcoli matematici: La tokenizzazione dei numeri è spesso arbitraria. Se l'algoritmo divide il numero 10045 nei token [10] e [045], ma il numero 20045 nei token [200] e [45], la rete neurale fatica ad apprendere le regole dell'aritmetica, poiché la rappresentazione numerica varia in base al contesto.
- Disparità tra lingue diverse: I tokenizer moderni vengono creati analizzando dataset prevalentemente in lingua inglese. Di conseguenza, un testo in inglese richiede pochissimi token (spesso 1 token per parola), mentre un testo in italiano, arabo o giapponese viene frammentato in molti più token. Ciò rende l'elaborazione di lingue diverse dall'inglese più lenta, più costosa sulle API e penalizzata nella finestra di contesto utilizzabile.
- Limiti della finestra di contesto e costi: La memoria di lavoro di un LLM (context window) e i relativi costi di elaborazione non si calcolano in parole o caratteri, ma in token. L'efficienza dell'algoritmo determina direttamente quanta informazione reale possa rientrare nel contesto del modello.
- "Glitch Token" e comportamenti imprevisti: Poiché il vocabolario viene estratto in modo statistico da dati grezzi presi da internet, può includere sequenze anomale (es. nomi utente Reddit, stringhe di codice ripetute o spaziature insolite). Quando un LLM incontra questi token "strani", il suo vettore di significato può destabilizzarsi, provocando risposte incoerenti o allucinazioni.

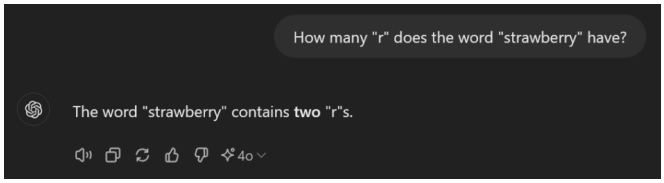
Tokenizer: Not a Technical Detail – ii

The Strawberry Problem 🍓

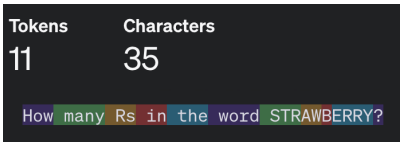
- Counting «r» in «Strawberry»



Claude 3.5 Opus



GPT4o



The Tokenizer Playground

Experiment with different tokenizers (running locally in your browser).

gpt-4 / gpt-3.5-turbo / text-embedding-ada-002

[demo]

Come nascono gli "Under-Trained Tokens"
- Dati di partenza "rumorosi": Se per costruire il vocabolario del tokenizer si utilizzano dati grezzi e non filtrati, l'algoritmo statistico individuerà stringhe o nomi utente che compaiono con frequenza anomala.
- Creazione del token: L'algoritmo di tokenizzazione assegna a questa stringa un ID numerico dedicato (un singolo token invece di dividere la parola in lettere o sottoparole).
- Mancanza di addestramento neurale: Durante l'addestramento vero e proprio dell'LLM, quel token specifico appare pochissime volte o solo in contesti privi di valore semantico. Di conseguenza, i pesi del vettore associato a quel token (embedding) rimangono incompleti o vicini allo zero.
- Il comportamento "Glitch": Quando un utente inserisce un glitch token nel prompt, la rete neurale riceve un vettore numerico che non sa come interpretare. Questo causa comportamenti imprevisti, come risposte prive di senso, allucinazioni, ripetizioni infinite o il rifiuto di rispondere.

sono delle stringhe di testo che entrano a far parte del vocabolario di un modello di linguaggio durante la fase di tokenizzazione, ma che la rete neurale non impara mai a gestire correttamente durante l'addestramento principale.

Under-Trained Tokens

- Many tokenizers have under-trained or glitch tokens
  - Do not trust noisy training data such as unfiltered Reddit forums
- A while back, people noticed some strange tokens in GPT-J (~GPT-2)
  - `« RandomRedditorWithNo»`
  - `«davidjl»`
  - `« SolidGoldMagikarp»`
  - `«BuyableInstoreAndOnline»`

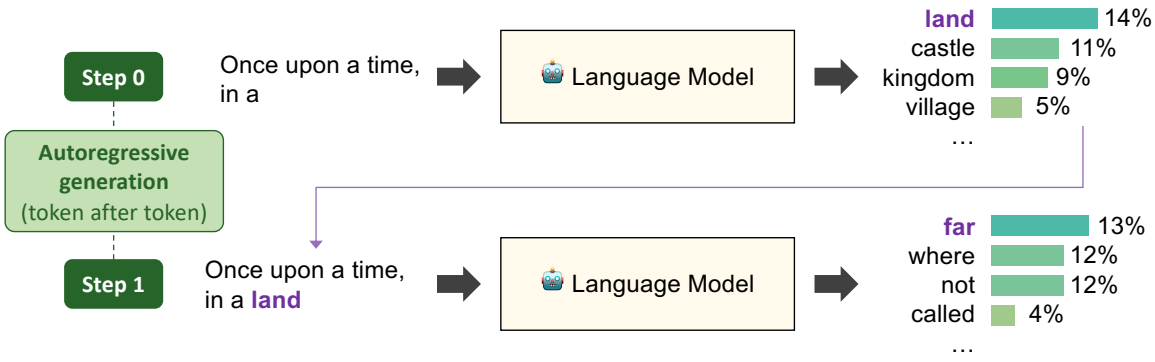
- StarCoder2 [Lozhkov et al., 2024] has a public dataset for tokenizer training
  - It contains our personal favourite token of all time

```
ittrLoremipumdolorsitametconsecteturadipiscingelitIntegervelvel
```

Seq2Seq: Where It All Begins

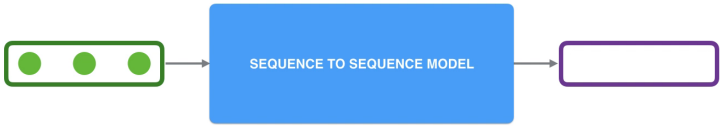
Generative NLP

- Autoregressive text generation is a multi-classification problem!
  - Predict the most probable next token over a vocabulary, i.e. each token is a class
  - Therefore, the output of a generative language model is a probability distribution



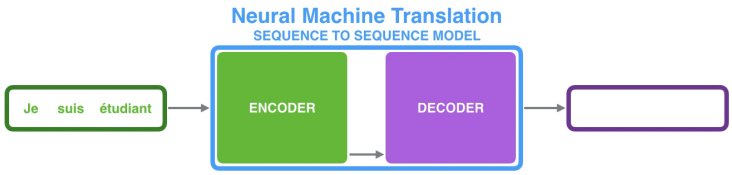
Seq2Seq: Introduction

- Sequence-to-sequence (seq2seq) models
  - A seq2seq model takes a sequence of items (e.g., words, letters, image features) and outputs another sequence of items
  - A seq2seq model lets inputs and outputs to be of different sizes
- Successful in generative tasks such as Machine Translation *(driving field)*, Text Summarization, and Image Captioning
  - 2 pioneering papers [Sutskever et al., 2014, Cho et al., 2014]; a revelation at the time
  - Google Translate started using them in production in late 2016 [Source]



Seq2Seq Architecture

- A seq2seq model is composed of an **encoder** and an autoregressive **decoder**
  - The encoder processes the input sequence one step at a time, and compiles the information it captures into a **context** vector
  - After processing the entire input sequence, the encoder sends the context over to the decoder, which begins producing the output sequence item by item
  - Note: the **context** vector has **fixed length memory** independently from the input length
  - The encoder and the decoder are usually implemented with **recurrent neural networks (RNNs)**, where Long Short-Term Memories (LSTMs) and Gated Recurrent Units (GRUs) help to avoid vanishing gradients



Seq2Seq Architecture: Encoder-Decoder Origin

- A seq2seq model is composed of an **encoder** and an autoregressive **decoder**

Clear role distinction in modeling and generation

"Let's first model the entire input sequence and then decide how to generate the output"

Seq2Seq Architecture: Generate After Input Reading

- A seq2seq model is composed of an **encoder** and an autoregressive **decoder**
  - The encoder processes the input sequence one step at a time, and compiles the information it captures into a **context** vector

- **In the past**, machine translation was phrase-based, i.e., the **source sentences were broken down into multiple chunks, and each chunk was translated independently, leading to output disfluency**
- This phrase-based method did not consider the meaning and context of the entire source sentence; it was not quite like how we, humans, translate
- **In contrast, a vanilla Seq2Seq model mimics the actions of (i) reading the entire source sentences, (ii) understanding its meaning, (iii) then producing a translation**

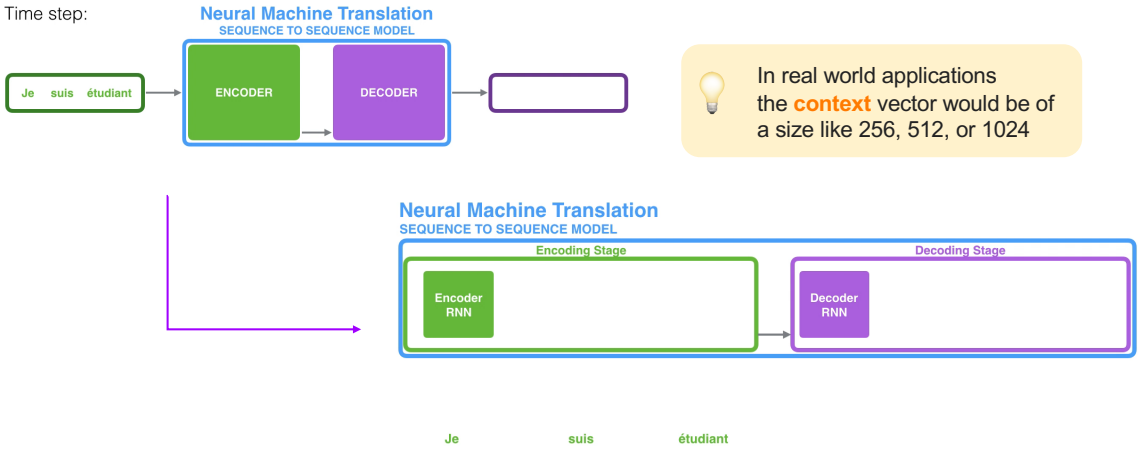
Seq2Seq Architecture: Encoder and Decoder are RNNs

- A seq2seq model is composed of an **encoder** and an autoregressive **decoder**
  - The encoder and the decoder are usually implemented with **recurrent neural networks (RNNs)**, where LSTMs and GRUs help to avoid vanishing gradients

- A seq2seq model can have different modifications
- The RNN models differ in terms of:
  a) **Directionality** – unidirectional or bidirectional
  b) **Depth** – single- or multi-layer
  c) **Type** – often either a vanilla RNN, a LSTM, or a GRU
- More information about RNNs and LSTMs on this blog post

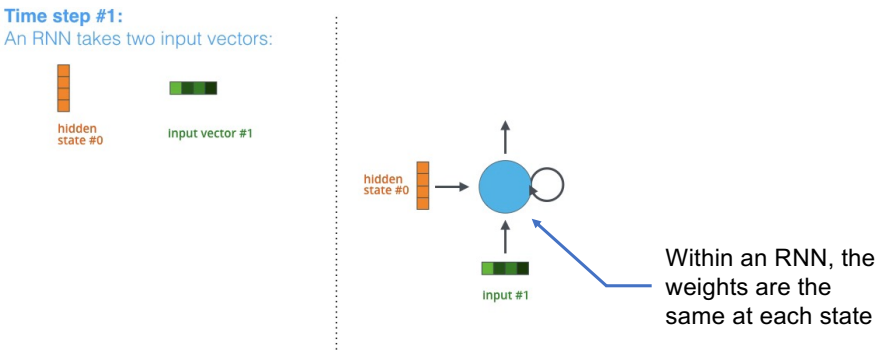
Seq2Seq Architecture: Unrolled View

- Unrolled visualization of a seq2seq model (build-then-deconstruct)



Seq2Seq Architecture: RNN Mechanics

- Hidden states depend on an earlier state and an input

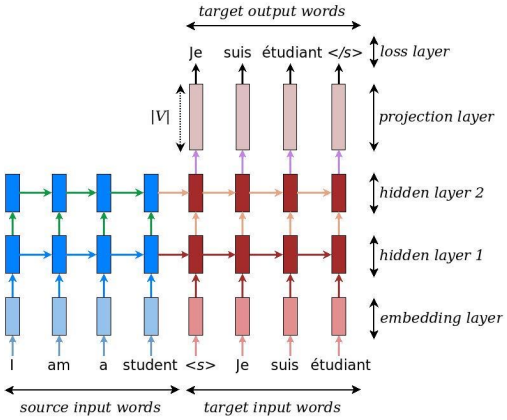


Note 1: The two RNNs in a seq2seq model, in principle, can share the same weights; however, in practice, we often use two different RNN parameters (such models do a better job when fitting large training datasets)

Note 2: the last encoder hidden state is actually the context we pass to the decoder

Seq2Seq Architecture: Multi-Layered RNNs

- Example of a deep multi-layer RNN-based architecture
  - Here, <s> marks the start of the decoding process, while </s> tells the decoder to stop



Seq2Seq Training: Cross-Entropy Loss

- Seq2seq models are trained to predict probability distributions of the next token given previous context (input x and previous output tokens y)
  - We have a training instance with the source $x = x_1, \dots, x_m$ and the target $y = y, \dots, y_n$
  - At the timestep t, a model predicts a probability distribution $p^{(t)} = p(* \,|y_1, \dots, y_{t-1}, x_1, \dots, x_m)$
  - The target at this step is $p^* = \text{one-hot}(y_t)$, i.e., we want a model to assign probability 1 to the correct token, y_t, and zero to the rest
- Cross-entropy loss for the target distribution p^* and the predicted distribution p

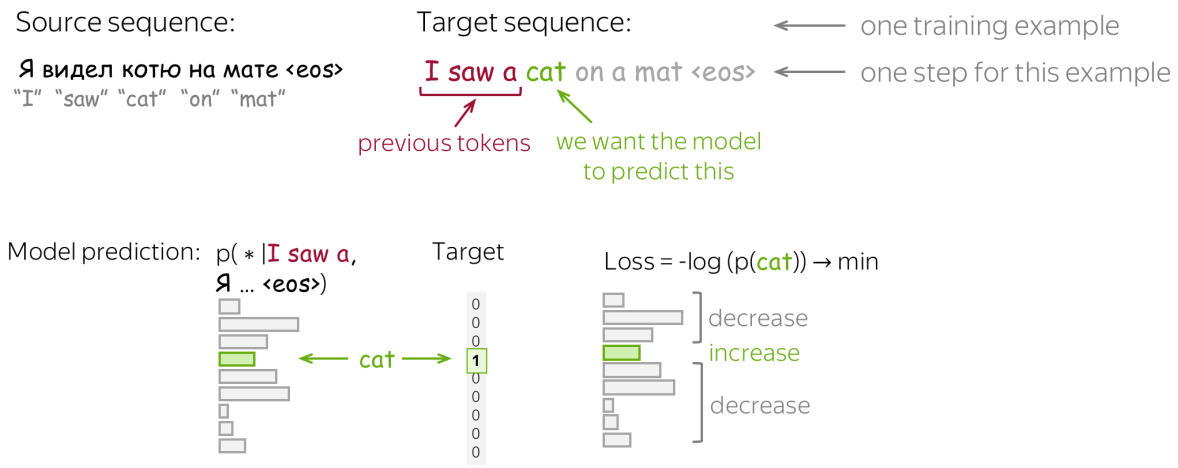
$$Loss(p^*, p) = -p^* \log(p) = -\sum_{i=1}^{|V|} p_i^* \log(p_i)$$

Since only one of p_i^* is non-zero (for the correct token y_t), we will get

$$Loss(p^*, p) = -\log(p_{y_t}) = -\log(p(y_t|y_{<t}, x))$$

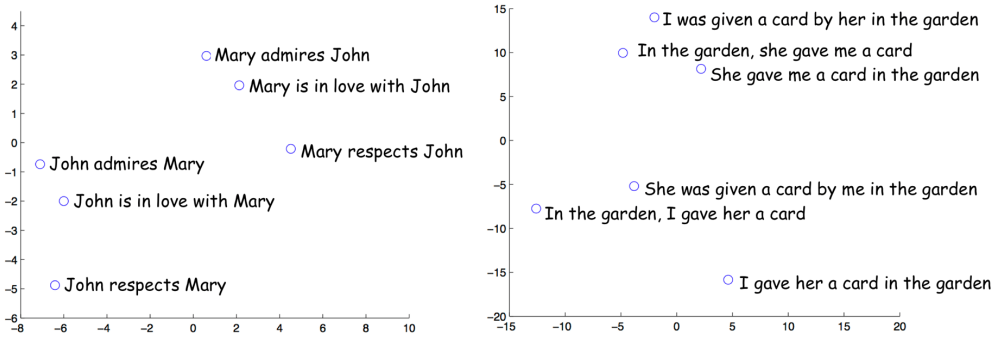
For the whole example: $$-\sum_{t=1}^{n} \log(p(y_t|y_{<t}, x))$$

Seq2Seq Training: Illustration of a Time Step



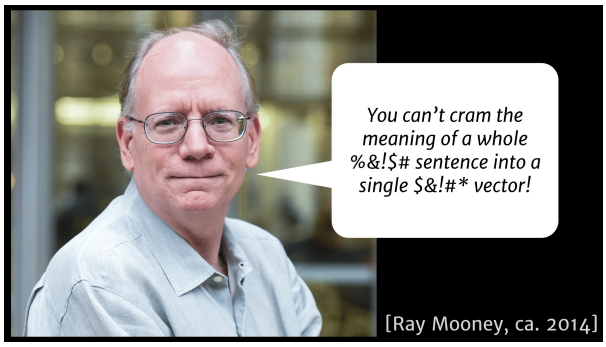
Seq2Seq: Visualizing Encoder Embeddings

- Sutskever et al. (2014) looked at the last encoder state and visualized several examples
  - interestingly, sentence representations with similar meaning but different structure are close



Information Bottleneck

- **The problem of fixed-size context vector (memory)**
  - (i) for the encoder, it is hard to compress all the information from the sequence; the number of possible source sequences (hence, their meanings) is $\infty \rightarrow$ it is likely to forget something
  - (ii) for the decoder, different information may be relevant at different steps
  - As sequence size increases, model performance slumps (also with LSTMs and GRUs)



Let's Pay Attention: Give Some Inputs More Weight

- 💡 Break down the input representation into a sequence of hidden states…
  - We could hold onto each token vector with its individual information instead of trying to smash them all into one big vector
- … But such a solution still has obvious flaws with memory and context

How to build a time and memory efficient model that predicts accurately from a long sequence?

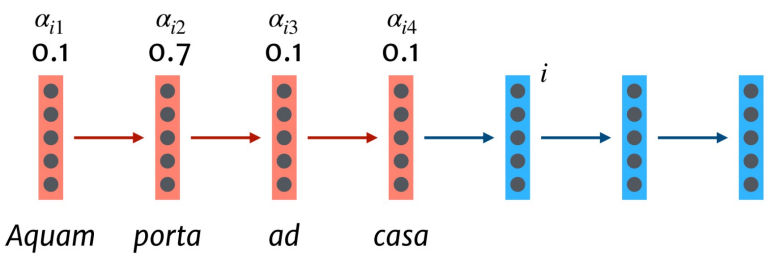
Maintaining all token hidden states might work if the model had a way of selecting and focusing on the likeliest input tokens each step

Seq2Seq + Attention: More Data, More Decoder Operations

- 👁️ *Not all input tokens are equally important for a certain output token; some deserve more attention than others*
- The **attention mechanism** is ubiquitous in modern deep learning models
  - Introduced and refined by [Bahdanau et al. 2014; Luong et al., 2015] for machine translation
- 💡 Add learnable attention layers to help the model **understand the weight of each input token at each decoder step** and as needed
  - The attention mechanism is part of a neural network and **allows the decoder to «look back»** at the complete input and **retrieve helpful information**
  - The ability to **amplify the signal from the relevant part of the input sequence** makes the attention models produce better results than models without attention
- In summary, at each time step...
  - The **encoder** passes a lot more data to the **decoder** (i.e., all the RNN hidden states instead of the last one)
  - An **attention-empowered decoder** does extra operations before producing its output

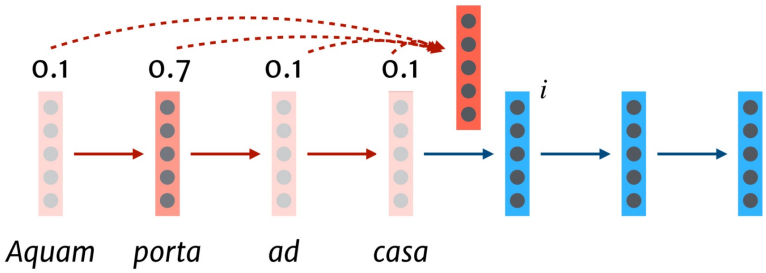
Seq2Seq + Attention: The Decoder Operations – i

1. When predicting output i, assign a weight α_{ij} to each encoder state h_j (weights sum to 1)



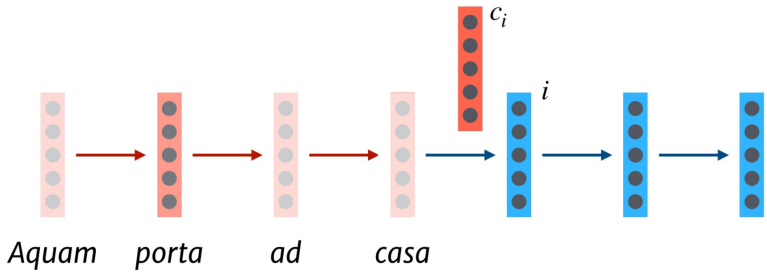
Seq2Seq + Attention: The Decoder Operations – ii

1. When predicting output i, assign a weight α_{ij} to each encoder state h_j (weights sum to 1)
2. Compute a pooled input $c_i = \sum_j \alpha_{ij} h_j$ («attention output» or «context vector»)



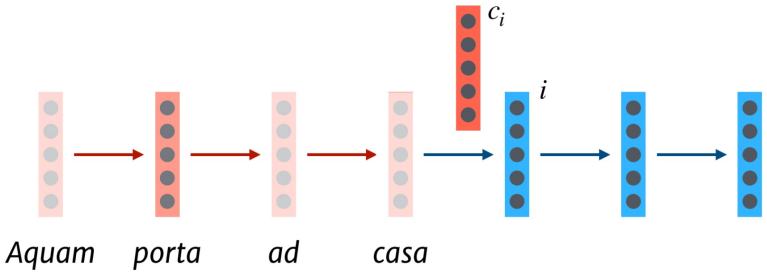
Seq2Seq + Attention: The Decoder Operations – iii

1. When predicting output i, assign a weight α_{ij} to each encoder state h_j (weights sum to 1)
2. Compute a pooled input $c_i = \sum_j \alpha_{ij} h_j$ («attention output» or «context vector»)
3. Use c_i to update the decoder parameters



Seq2Seq + Attention: The Decoder Operations – iv

1. When predicting output i, assign a weight α_{ij} to each encoder state h_j (weights sum to 1) *... How?*
2. Compute a pooled input $c_i = \sum_j \alpha_{ij} h_j$ («attention output» or «context vector»)
3. Use c_i to update the decoder parameters *... How?*



Seq2Seq + Attention: How to Compute Attention Weights

- Three popular ways to implement the attention score function
  - h_t = decoder state at step t; s_k = encoder state for token k

Dot-product

$h_t^T \times s_k$

$\text{score}(h_t, s_k) = h_t^T \, s_k$

Element-wise Multiplication
The simplest method *(computational efficient but limited in expressiveness)*

Multi-Layer Perceptron

$w_2^T \times \tanh\left[W_1 \times \begin{bmatrix} h_t \\ s_k \end{bmatrix} \right]$

$\text{score}(h_t, s_k) = w_2^T \cdot \tanh(W_1 [h_t, s_k])$

Concatenation + Feed-forward layer w/ tanh activation function W_1 + weight vector w_2
«**Bahdanau** attention» [Bahdanau et al., 2014], the paper that presented the attention for the first time, co-authored by Yoshua Bengio *(expressive but computational expensive)*

Bilinear

$h_t^T \times W \times s_k$

$\text{score}(h_t, s_k) = h_t^T \, W s_k$

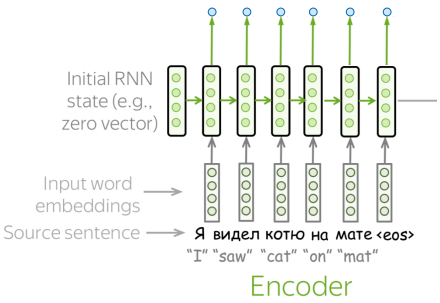
«**Luong** attention» [Luong et al., 2015]
Compared to dot-product, it adds a weight matrix W that allows learning more complex relations *(middle ground)*

Seq2Seq + Attention: How to Use the Attention Output

- The attention output c_i is combined with the current decoder state to produce an **attentional hidden state**
  - This is usually done through concatenation followed by a linear transformation
  - E.g., $\text{attentional state} = \tanh(\boldsymbol{W}[c_i, h_t])$
- The attentional hidden state is then used to predict the output token for the current time step, typically through a softmax layer
- The predicted output is compared to the actual target output to compute the cross-entropy loss
- The loss is propagated through the network for **parameters update**, including:
  - **Attention mechanism** ($\boldsymbol{W_1}$ and $\boldsymbol{w_2}$ in Bahdanau, $\boldsymbol{W}$ in Luong)
  - Encoder
  - Decoder
- **Key point:** the attention mechanism becomes an integral part of the network

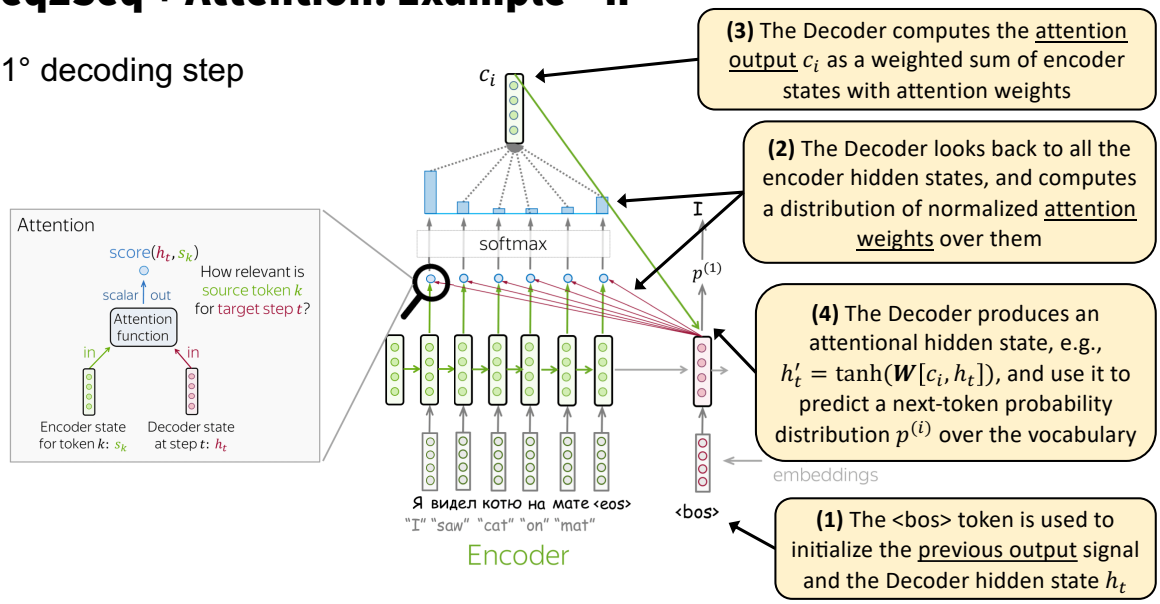
Seq2Seq + Attention: Example – i

- Full encoded input



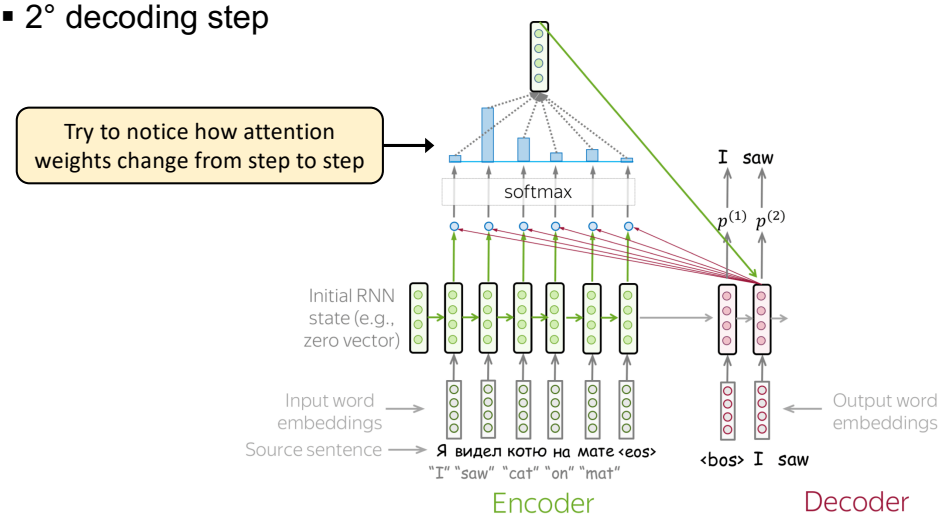
Seq2Seq + Attention: Example – ii

- 1° decoding step



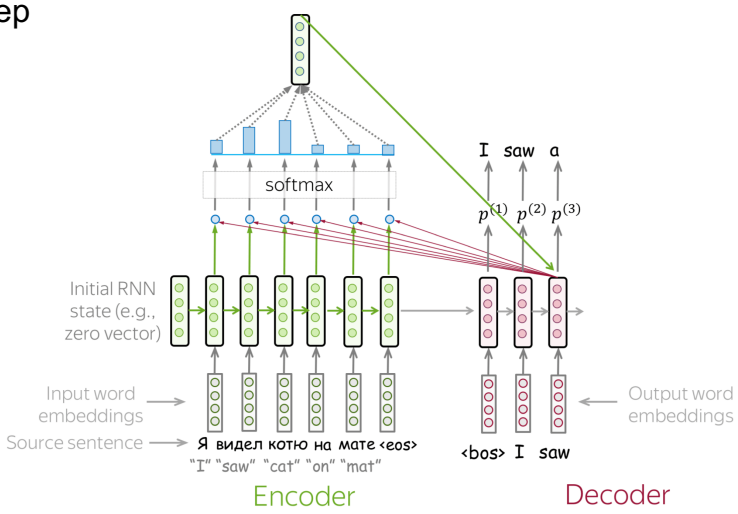
Seq2Seq + Attention: Example – iii

- 2° decoding step



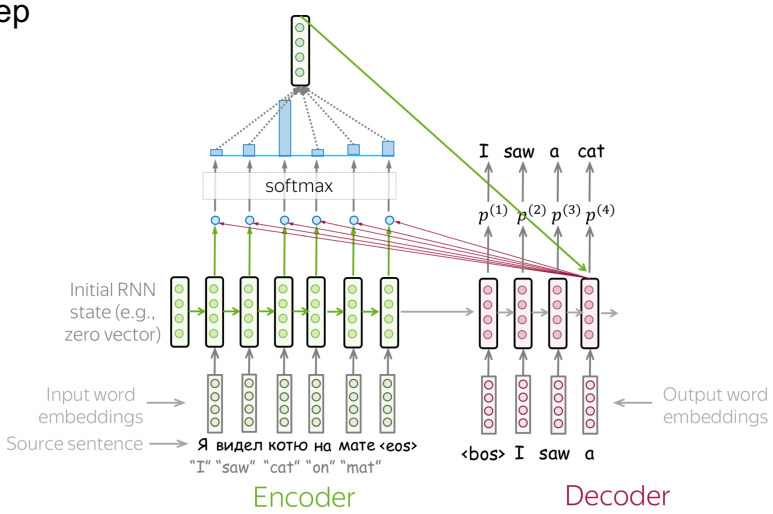
Seq2Seq + Attention: Example – iv

- 3° decoding step



Seq2Seq + Attention: Example – v

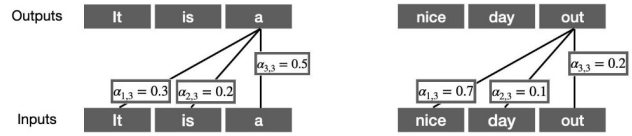
- 4° decoding step



Attention vs Fully-Connected Layers: A Common Question

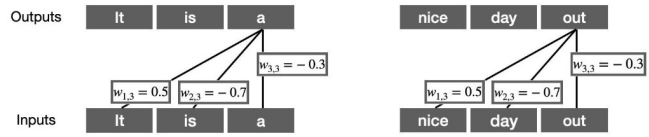
- How attention scores differ from weights in a fully-connected layer?
  - Conceptually, attention mechanisms allow to attend to different parts of a sequence. On the surface, this sounds very similar to a fully connected layer, indeed!

Attention scores α are dynamic with respect to the input



In attention mechanisms, attention weights are **dynamic** and **input-dependent**

Fully-connected layer weights w are static with respect to the input

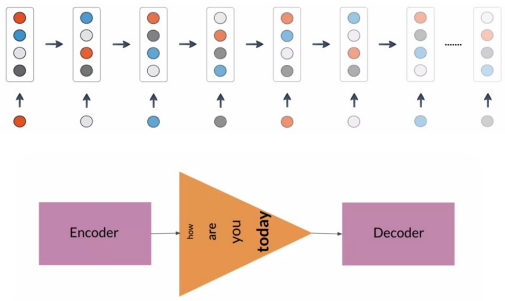


In contrast, the weights of a convolutional or fully-connected layer are **fixed after training**

✍ *As a side note: the attention can be implemented using fully-connected layers*

Seq2Seq Shortcomings: Decay of Information through Time

- Even with attention, there are some flaws...
- RNNs struggles with capturing **long-range dependencies** because of their sequential nature; i.e., the first input items are likely to have little effect on the distant final items



Seq2Seq Shortcomings: Efficiency

- Each hidden state depends on the output of the previous hidden state
- The sequential nature of the RNNs **precludes parallelization** within training examples, which becomes critical at longer sequence lengths (i.e., long processing time)
- Waiting for data to become available makes RNNs unfit with technologies like CuDNN, **slowing down the whole GPU process**
- Sequential processing also **limits batch sizes**, resulting in slower training times and increased memory requirements

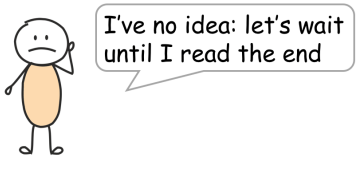
Seq2Seq Shortcomings: Monodirectional Token Encoding

- Seq2Seq models can produce different representations for tokens depending on their context; this is achieved through the recurrent nature of RNNs
- However, traditional Seq2Seq models process tokens from left to right
  - The encoder processes the input sequence in one direction
  - At each step, it only has information about the tokens it has seen so far; it cannot incorporate information from future tokens into the representation of the current one
  - Thus, it struggle to properly embed a token when relevant context comes after it

I arrived at the bank after crossing the ...

What does **bank** mean in this sentence?

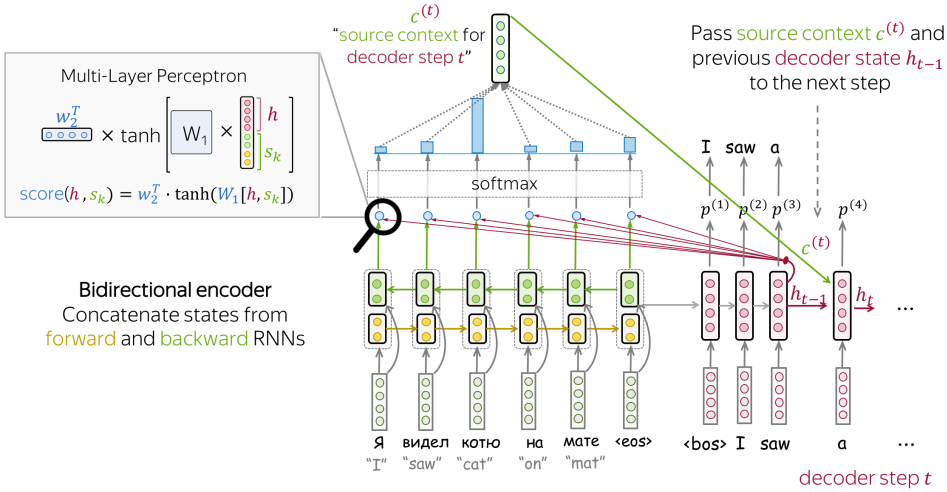
...street? ...river?



RNNs

Seq2Seq: Bidirectional RNNs as Encoder

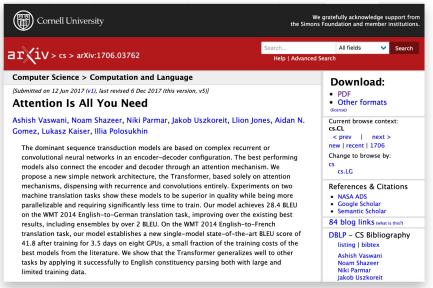
- Some works use two RNNs (left-to-right, right-to-left) and combine their output



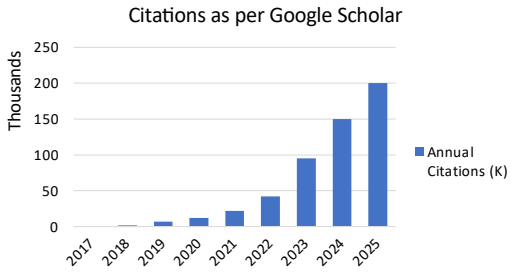
Transformers

Transformer: Attention Is All You Need

- The core invention underlying the current boom in AI
  - A Transformer is a deep learning architecture: the new language modeling technology
  - The original Transformer was introduced in 2017 by Google for the specific use case of translating text from one language into another (i.e., sequence transduction)
  - The publication made a lot of «noise» lately, progressively extending its impact from NLP to Computer Vision, Time Series Analysis, Reinforcement Learning, Protein Folding, etc.



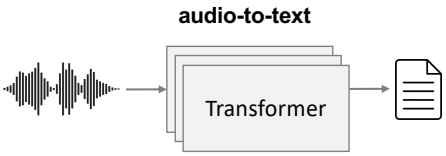
Attention Is All You Need [Vaswani et al., 2017]



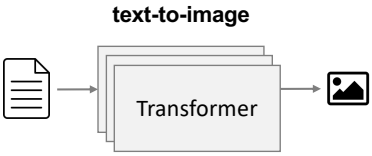
🔥 One of the most cited paper in AI history

Transformer: An Architecture for All Seasons

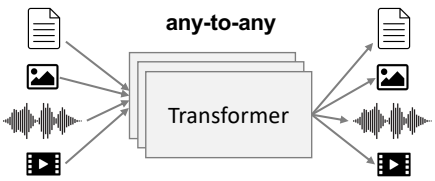
- Transformers can be used to build many kinds of generative models



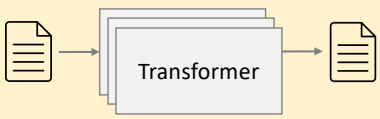
E.g., Whisper [Radford et al., 2022]



E.g., DALL-E 3 [OpenAI, 2023]



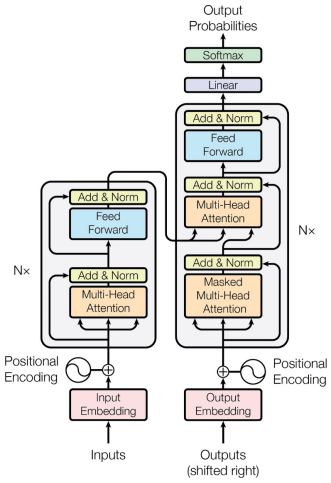
E.g., NExT-GPT [Wu et al., 2023]



The variant we will focus now is the more traditional, underlying the majority of today's LLMs

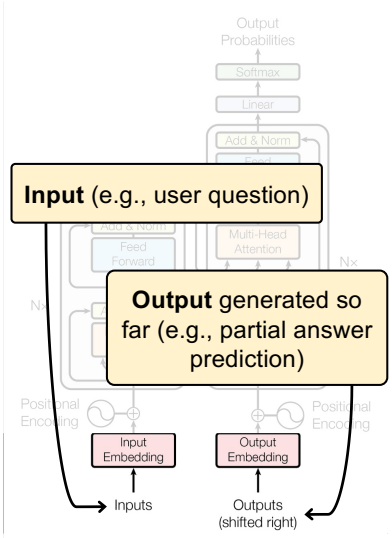
Transformer: Architecture

- The architecture behind almost every LLM

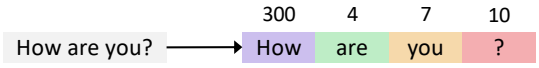


Let's understand this diagram brick by brick

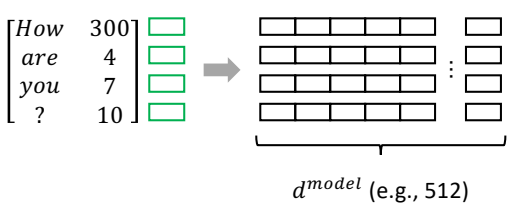
Input and Output Embedding



- Tokenization

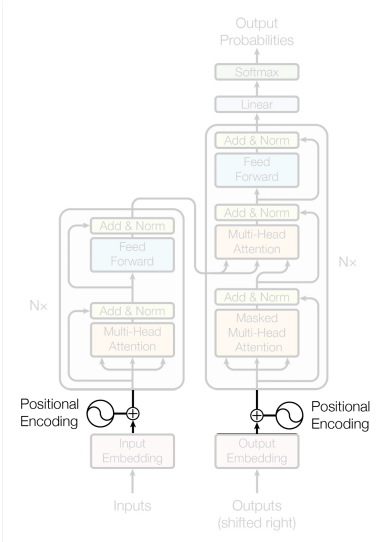


- Embedding layer
  - Each token is mapped to a high-dimensional vector using a trainable embedding layer (randomly initialized)



Extend the representative power

Positional Encoding



- Since a Transformer contains no recurrence and no convolution, we must inject positional information
  - E.g. «How are you ? Are you doing well ?»
  - The 2 are/you should be represented with distinct vectors
- Positional encoding (PE) **adds** a «position» vector of size d_{model} to each input token embedding
  - Incorporate a time signal into the input
  - It gives insight into the position of tokens within a sequence and the relative distances between them

How	are	you	?
↓	↓	↓	↓
Embedding (How)	Embedding (are)	Embedding (you)	Embedding (?)
+	+	+	+
Positional (How)	Positional (are)	Positional (you)	Positional (?)

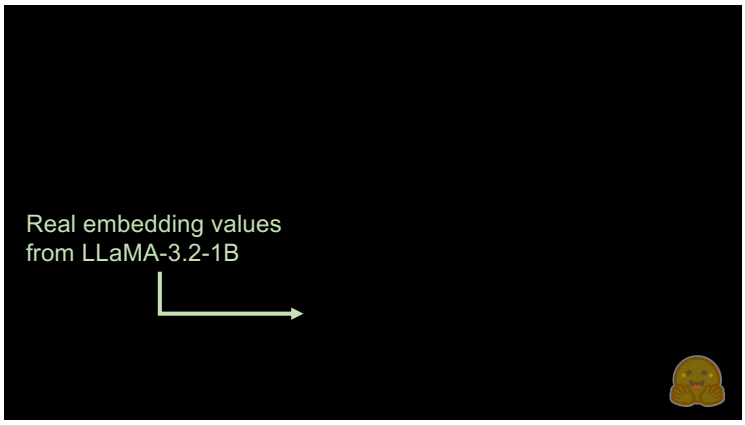
Positional Encoding: Desirable Properties

PE can be learned or be a pre-defined function (most common)

1. **Unique encoding for each position (across sequences)**
   - Each position needs a unique encoding that remains constant regardless of seq. length
   - Token at position 5 → same encoding whether the sequence is of length 10 or 10,000
2. **Linear relation between two encoded positions**
   - If we know the encoding for position p, it should be straightforward to compute the encoding for position $p + k$, making it easier for the model to learn positional patterns
3. **Generalizes to longer sequences than those encountered during training**
   - To increase our model's utility in the real world, our encoding scheme needs to be adaptable enough to handle unexpected input lengths, w/o violating any other properties
4. **Generated by a deterministic process the model can learn**
   - A deterministic process to draw positional encodings would allow the model to learn the mechanism behind our encoding scheme efficiently

Integer Positional Encoding

- The first approach that may jump to mind is simply to add an integer value to the token position, with values ranging from 0 to L (sequence length)

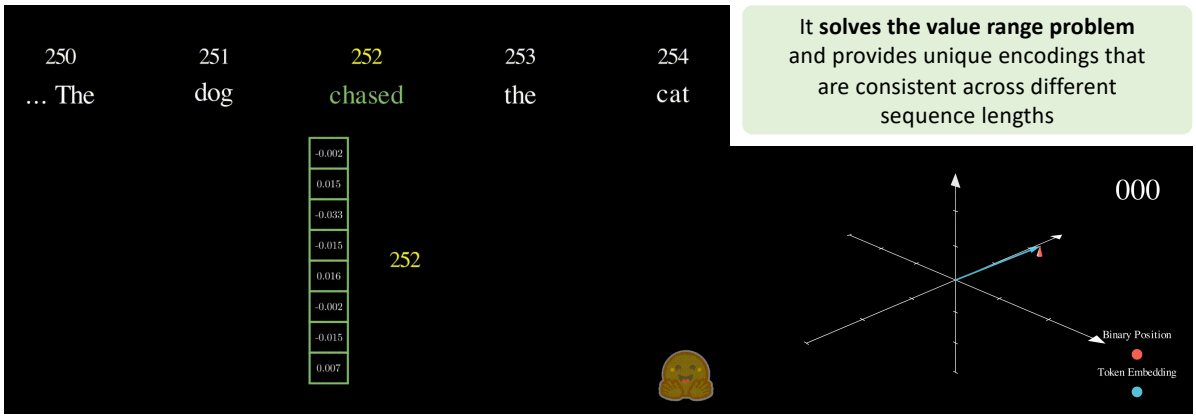


Embedding values are generally clustered around 0; this is desirable to avoid vanishing or exploding gradients during training

The magnitude of the position value vastly exceeds the actual values of our input: the **signal-to-noise ratio is very low**, and it is hard for the model to separate the semantic information from the positional information

Binary Positional Encoding

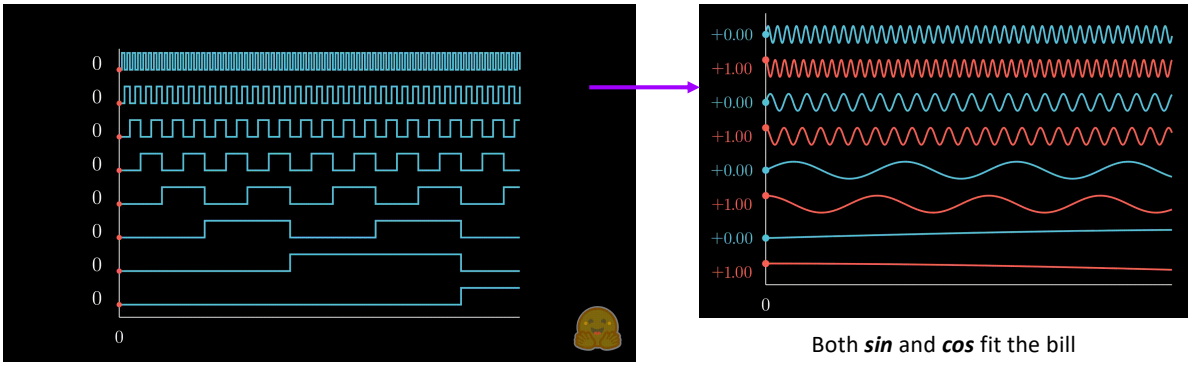
- Convert a position (e.g., 252) into its binary representation (e.g., 11111100), and add each bit to the corresponding component of the token embedding



It **solves the value range problem** and provides unique encodings that are consistent across different sequence lengths

Binary Positional Encoding: Issues

- Very «jumpy» result (as we might expect from the discrete nature of binary)
- The optimization process prefers smooth, continuous, and predictable changes
  - Do you know any functions with similar value ranges that are smooth and continuous ?



Both ***sin*** and ***cos*** fit the bill

Sinusoidal Positional Encoding

- Alternate sine and cosine functions of varying frequencies

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10{,}000^{\frac{2i}{d_{model}}}}\right) \quad PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10{,}000^{\frac{2i}{d_{model}}}}\right)$$

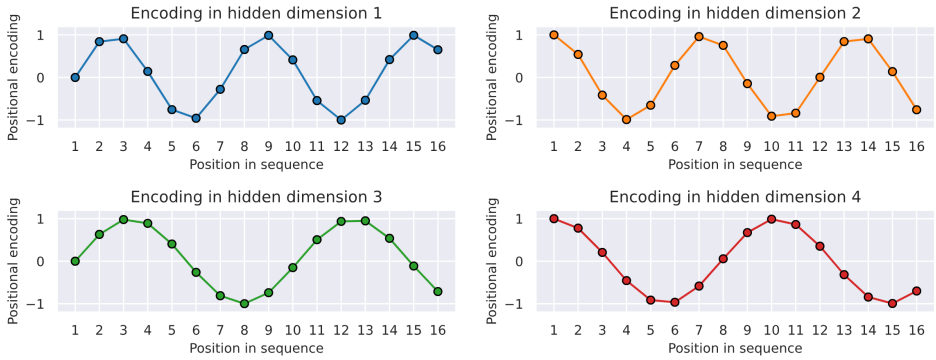
Example: $d_{model} = 256$

	pos	idx=0	idx=1	idx=2	...	idx=255
How	0	sin(0/1) = 0	cos(0/1) = 1	$\sin(0/10000^{1/256})$ = 0		$\cos(0/10000^{254/256})$ = 1
are	1	sin(1/1) = 0.84	cos(1/1) = 0.54	$\sin(1/10000^{1/256})$ = 0.82		$\cos(1/10000^{254/256})$ = 0.99..901
you	2	sin(2/1) = 0.91	cos(2/1) = -0.42	$\sin(2/10000^{1/256})$ = 0.93		$\cos(2/10000^{254/256})$ = 0.99..604
?	3	sin(3/1) = 0.14	cos(3/1) = -0.99	$\sin(3/10000^{1/256})$ = 0.24		$\cos(3/10000^{254/256})$ = 0.99..112

Roughly gives decreasing importance to higher latent dimensions in distinguishing the token positions

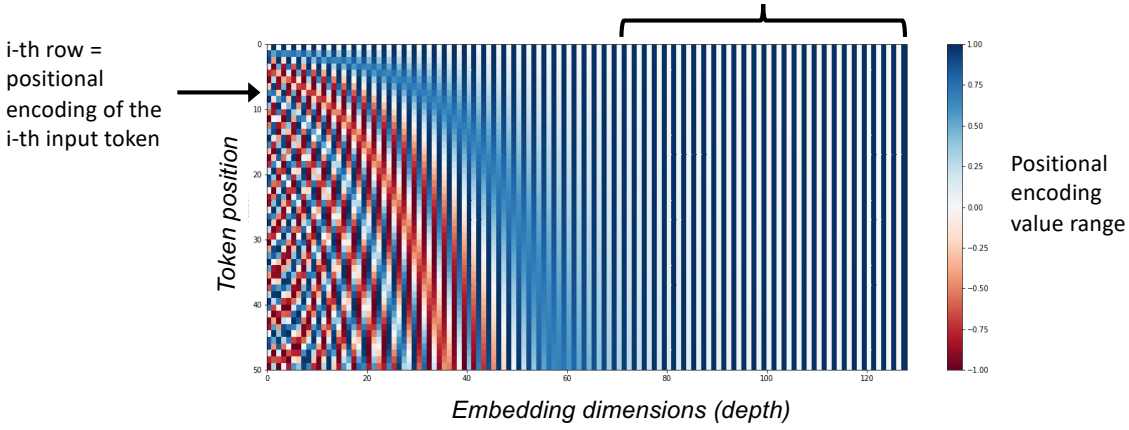
Sinusoidal Positional Encoding: Univocity

- Output in $[-1,1]$ keeps the values of the PE matrix in a normalized range
- As the sinusoid for each position is different, you have a unique way of encoding each position
  - One can quantify the similarity among different positions (encode relative positions)



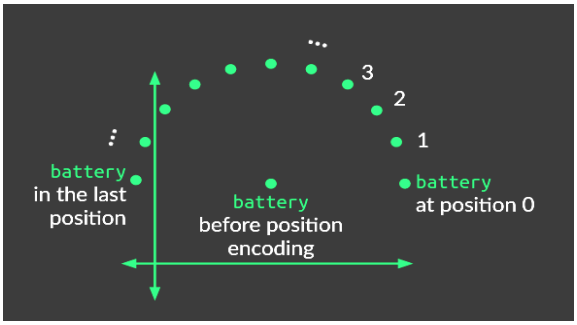
Sinusoidal Positional Encoding: Visualization

In higher dimensions, here after the 70th, the values tend to alternate similarly and thus contribute less to distinguishing token positions.
However, when combined, all dimensions collectively serve their purpose.



Sinusoidal Positional Encoding: Circular Wiggle Intuition

- The position of the token in the embedding space acts as the center of a circle
- A perturbation is added to it, depending on where it falls in the sequence
  - For each position, the token is moved in the same distance but at a different angle, resulting in a circular pattern as you move through the sequence
  - Words that are close to each other in the sequence have similar perturbations, but words that are far apart are perturbed in different directions



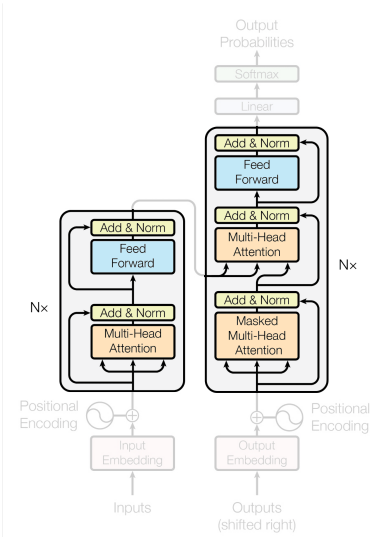
Observations

To **prevent increasing the number of parameters**, positional embeddings are **added** (not concatenated)

✗ Sinusoidal encodings embed **absolute** positions and not **relative** distances between input tokens

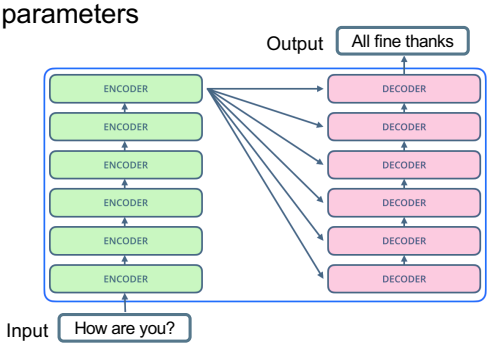
✗ Similarity relations between the same token pairs are **lost** when they are positioned differently in the input sequence - because of the different absolute distance

Encoder + Decoder

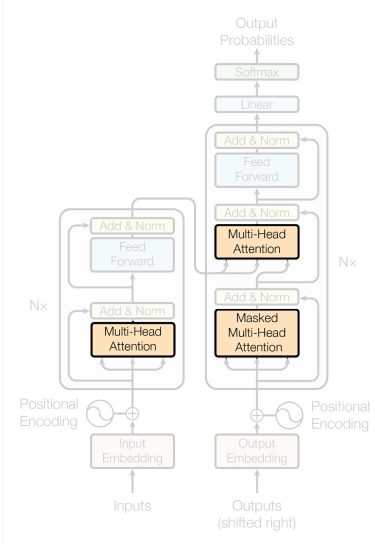


- Similar to seq2seq, the architecture follows an encoder-decoder structure
  - The Transformer consists of multiple (originally N=6) encoder and decoder layers
  - The last encoder layer propagates to all decoder layers
  - All encoder/decoder layers have identical structure but learn separate parameters

Architecture composition
=
manipulation steps on top of the input/output representations



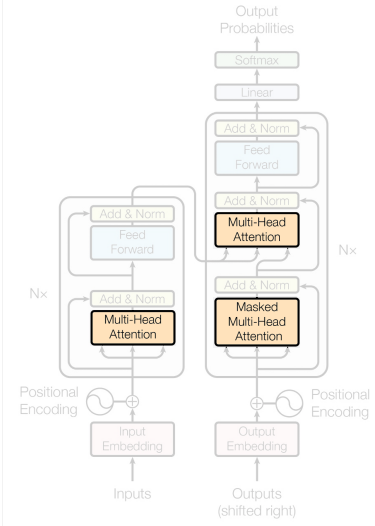
A New Attention-Based Modeling Paradigm – i



- In contrast to previous models where processing within encoder and decoder was done with recurrence or convolutions…
- … Transformers operates using only attention

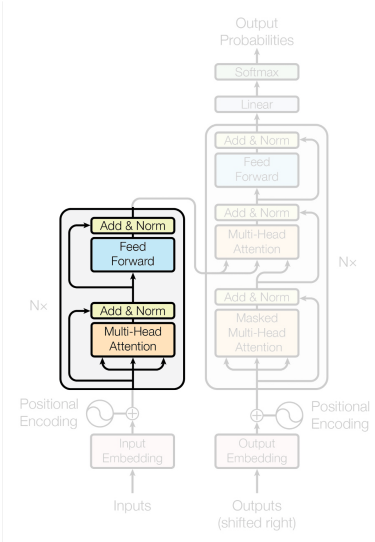
	Seq2seq w/o attention	**Seq2seq w/ attention**	**Transformer**
Processing within encoder	RNN/CNN	RNN/CNN	Attention
Processing within decoder	RNN/CNN	RNN/CNN	Attention
Decoder-encoder interaction	Static fixed-sized context vector	Attention	Attention

A New Attention-Based Modeling Paradigm – ii

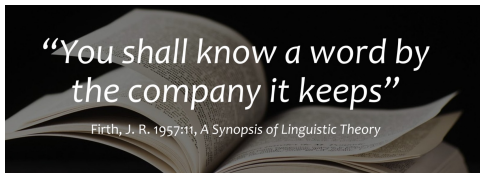


Vaswani et al. **Attention is all you need**, NIPS 2017.

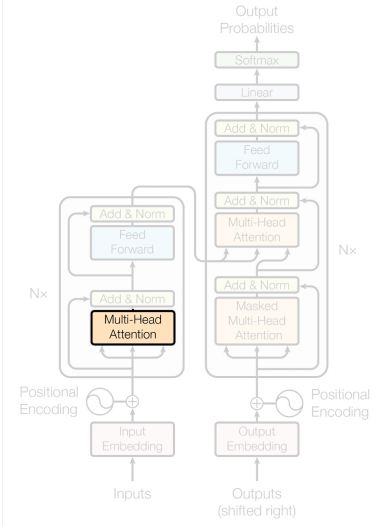
Encoder



- Each encoder layer produces a latent representation for each input token
- **Intuition**
  - Transformer's encoder can be thought of as a sequence of reasoning steps (**layers**) to capture the input meaning
  - At each step, input tokens look at each other to evolve their vectorial representation (information exchange)
  - Understand each other in the whole input context



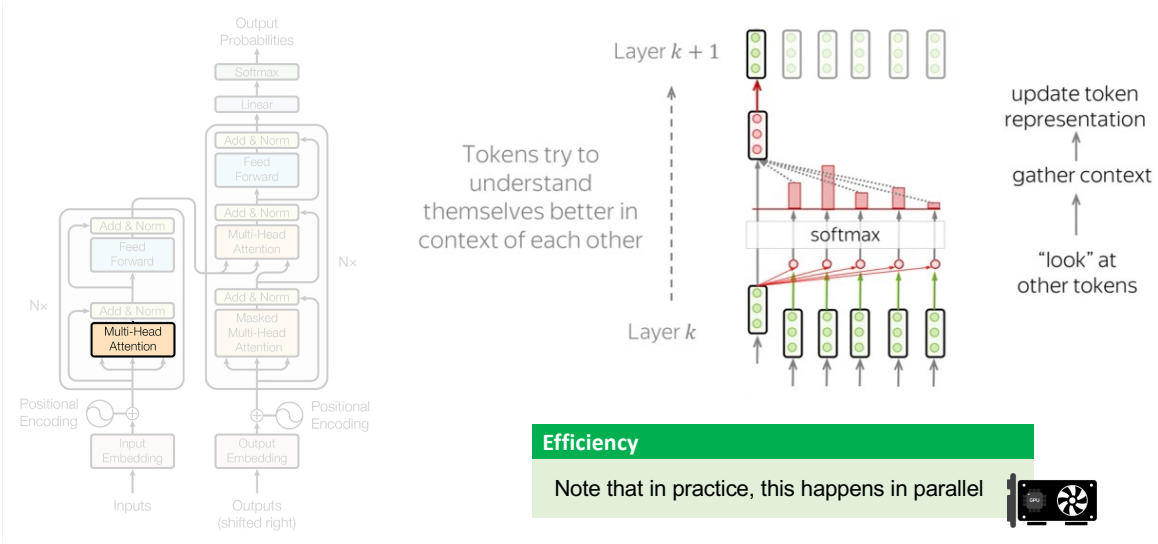
Self-Attention: Differences with Seq2Seq Attention



- Unlike seq2seq attention, self-attention operates between representations of the same nature
  - Encode each token based on all the other tokens in the *same* sequence (**bidirectional surrounding context**)

Attention type	**Is looking**	
	From	**At**
Decoder-encoder attention (a.k.a. cross-attention)	One current decoder state	All encoder states
Self-attention	Each state from a set of states (i.e., one encoder or decoder layer)	All other states in the same set

Self-Attention: Illustration

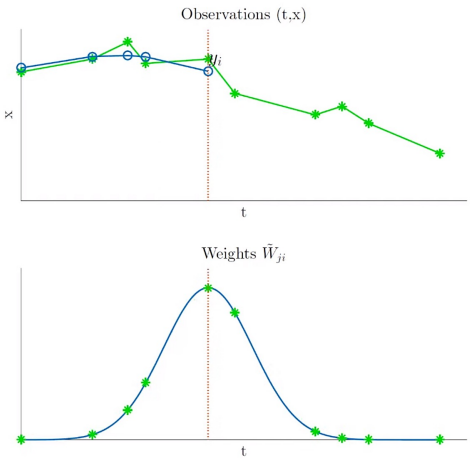


Efficiency

Note that in practice, this happens in parallel

Self-Attention: A Weighted Average (Scalar)

- Attention is about computing **weighted averages** of «related» values
- A **scalar example** to gain intuition
  - From noisy observations $\langle t_j, x_j \rangle$ of some unknown distribution, we need to estimate y_i for a certain data point t_i of the underlying distribution
    - E.g. $\langle t_i, x_i \rangle$ is a temperature x_i sensed at time t_i, and y_i is the temperature to be estimated from all other observations
- A possible strategy: assigning weights based on a relationship of closeness to t_i
  - Gaussian kernel to give nearby data points larger weights (bell-shaped curve centered at t_i)



$$\widetilde{W}_{ji} = \mathcal{N}(t_j;\ t_i, 1) \qquad W_{ji} = \frac{\widetilde{W}_{ji}}{\sum_r \widetilde{W}_{ri}} \qquad y_i = \sum_j W_{ji} x_j$$

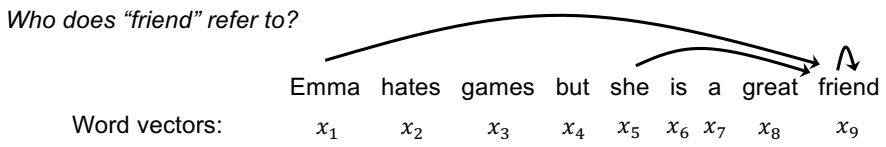
Gaussian distribution with a variance of 1 (weights decrease with $|t_j - t_i|$)

Weight normaliz. (sum 1)

Weight calculation

Self-Attention: A Weighted Average (Text)

- How can we apply weighted averages within the realm of **languages**?



- Example: update the "friend" vector ($x_9 \rightarrow y_9$)
  - Whereas the original token embedding for "friend" x_9 is generic and designed to make sense in any context that token might appear...
  - The new y_9 is adapted to the context, where the friend is a female person named Emma
  - 💡 Keep intact the values of the token(s) we want to focus on and drown-out the irrelevant ones by multiplying them by tiny weights (e.g., 0.001)

$$y_9 = \sum_i w_i x_i = w_1 x_1 + w_2 x_2 + \cdots + w_9 x_9$$

The engine of contextual understanding

Self-Attention: Need for Dynamic Weighting

- How to determine weights?
  - We are obviously not planning to select the weights by hand, and we need some mechanism for automatically computing them
- First idea for generating the new embedding for Emma x_9:
  - dot product

$$z_1 = x_1^T x_9, \quad z_2 = x_2^T x_9, \quad \ldots$$

$$[w_1, w_2, \ldots, w_9] = softmax(z_1, z_2, \ldots, z_9)$$

Similar words ⇒ Large weight

  - **Note 1:** the weights w_1 to w_9 are specific for the token "friend"
  - **Note 2:** the dot product is unbounded and can be negative; its value depends on the magnitudes and directions of the vectors involved ⇒ **softmax**: normalize attention weights into probabilities (sum 1)
  - ❌ We should not just give large weights to tokens with similar embeddings
  - ❌ There are no free parameters that can be adjusted to obtain better weights

Self-Attention: Learnable Weight Matrices (Projections)

- Each token's representation is linearly transformed into three roles
  - Attention is implemented with **Query (Q)**, **Key (K)**, **Value (V)** abstractions, a paradigm analogous to a database lookup (e.g., hashing functions)

Each vector receives three representations ("roles")

$W_Q \times = $ Query: vector **from** which the attention is looking

What we are looking for

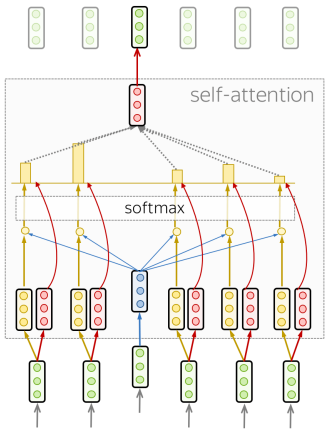
$W_K \times = $ Key: vector **at** which the query looks to compute weights

The features that the query interacts with to determine relevance

$W_V \times = $ Value: their weighted sum is attention output

The actual content to return

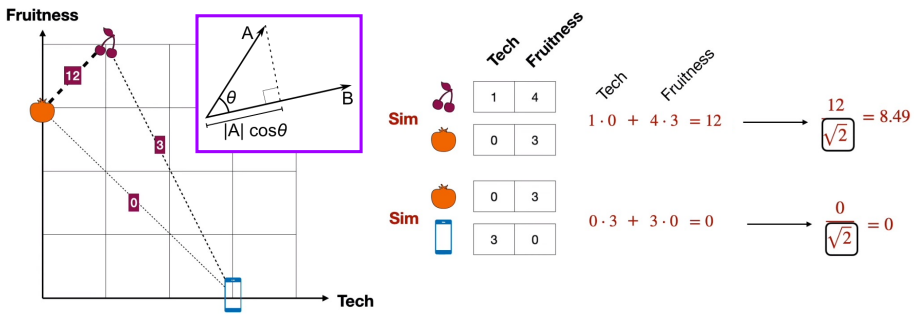
Projection matrices to be learned!



Any token can both:
- request information from others (via Q)
- supply information to others (via K, V)

Self-Attention: Scaled Dot Product

- The similarity measure actually used by self-attention is the **dot product** …
  - … **divided by the square root of the vector length** (i.e., scaled dot product) **WHY ?**
  - As the dimension ***d*** of vectors increases, the absolute value of their scalar product tends to grow according to the square root of ***d***
  - Hence, the softmax scores of a very long vectors (e.g., thousands of coordinates) are pushed into regions 0 or 1, potentially leading to **extremely small gradients**
  - To ensure numerical stability the scalar product must be scaled



Self-Attention Math

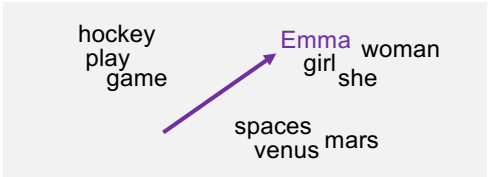
- To compute weights we first compute query q and key k vectors
  - The self-attention layers in a Transformer architecture introduce **two matrices W^Q and W^K** that are used to transform the vectors before computing the dot product
  - This is more flexible; **W^Q and W^K contain free parameters that we can train**
  - *Emma hates games but she is a great friend* (x_1 = «Emma» token embedding)

Query: $q_1 = W^Q x_1$ **Keys:** $k_1 = W^K x_1,\ k_2 = W^K x_2,\ \ldots,\ k_9 = W^K x_9$

$z_1 = k_1^T q_1/\sqrt{d},\ z_2 = k_2^T q_1/\sqrt{d},\ \ldots,\ z_9 = k_9^T q_1/\sqrt{d}$ $\quad [w_1, w_2, \ldots, w_9] = softmax(z_1, z_2, \ldots, z_9)$

Values: $v_1 = W^V x_1,\ v_2 = W^V x_2,\ \ldots,\ v_9 = W^V x_9$

New Emma embedding: $y_1 = \sum_{j=1}^{9} v_j w_j$



Attention Math: Matrix Formulation

Input: $X = [x_1, x_2, \ldots, x_n]$, $\mathrm{Q} = [q_1, q_2, \ldots, q_n]$, $\mathrm{K} = [k_1, k_2, \ldots, k_n]$, $\mathrm{V} = [v_1, v_2, \ldots, v_n]$

New output embeddings after computing the attention: $\boldsymbol{Y} = [y_1, y_2, \ldots, y_n]$

Matrix notation

- Calculations without for-loops:

Query: $Q = W^Q X$,
Keys: $K = W^K X$,
Values: $V = W^V X$,
$Z = K^T Q/\sqrt{d}$.

$= W^Q [x_1, x_2, \ldots, x_n]$
$= [W^Q x_1, W^Q x_2, \ldots, W^Q x_n]$
$\Longrightarrow q_1 = W^Q x_1,\ q_2 = W^Q x_2,\ \ldots$

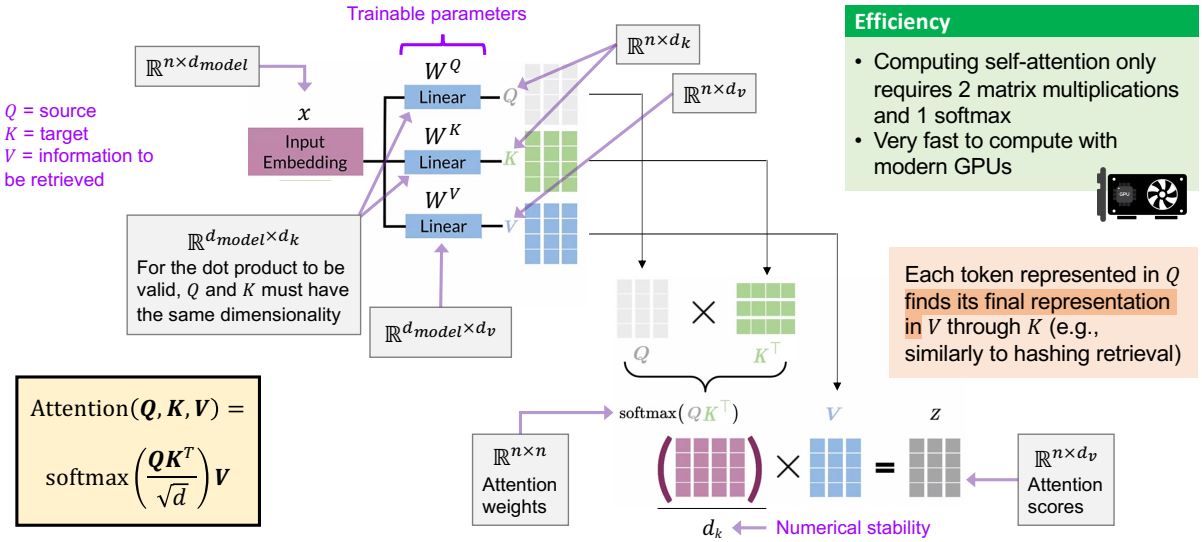
$z_{ji} = k_j^T q_i / \sqrt{d}$

- Weights and averages using column-wise softmax:

Weights: $W = softmax(Z)$ **New embedding:** $\boldsymbol{Y} = VW$

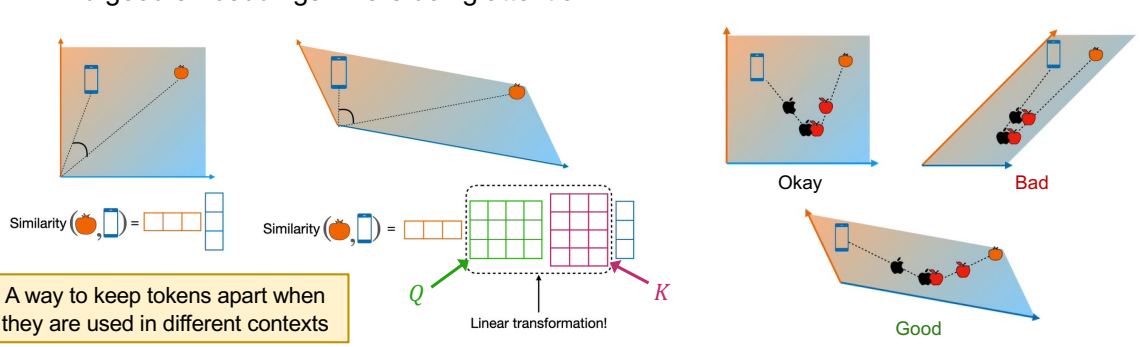
$y_i = V \begin{bmatrix} W_{1i} \\ W_{2i} \\ \vdots \\ W_{ni} \end{bmatrix} = \sum_{j=1}^{n} v_j W_{ji}$

Attention Math: Matrix Formulation (Graphic Illustration)



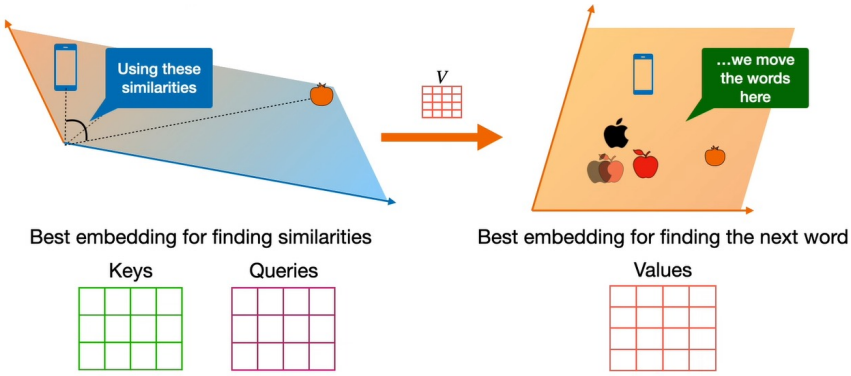
Self-Attention: Geometrical Intuition (Q and K)

- Learn transformation matrices to map input embeddings onto optimal spaces for properly assigning attention weights, namely just a space of their similarities
- W^Q and W^K work together to create **linear** transformations
  - Learnable transformation matrices used to project the input sequence in different spaces
  - Depending on their specifics, they will stretch, rotate or shear the original vectors to help us find good embeddings where doing attention



Self-Attention: Geometrical Intuition (V)

- The latent space defined by the product of W^Q and W^K does not model the token embeddings, but their **similarities**
  - Each cell i, j of QK^T contains the similarity between q_i and k_j (and not token embeddings)
  - W^V produces an extra linear transformation, turning it into an **embedding** space optimized for next token prediction, i.e., each resulting row is a d-dimensional token embedding



Is One Attention (One Interpretation) Enough?

- Usually, understanding the role of a word in a sentence requires understanding how it is related to different parts of the sentence
- Each word is part of many relations
- Therefore, we have to let the model focus on different things

"The bank will not approve the loan because it is too risky"

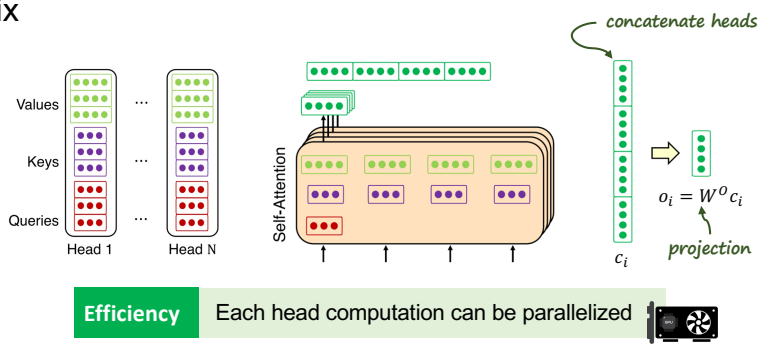
Syntax "The bank will not approve the **loan** because **it** is too risky"

Semantics "The **bank** will not approve the **loan** because it is too risky"

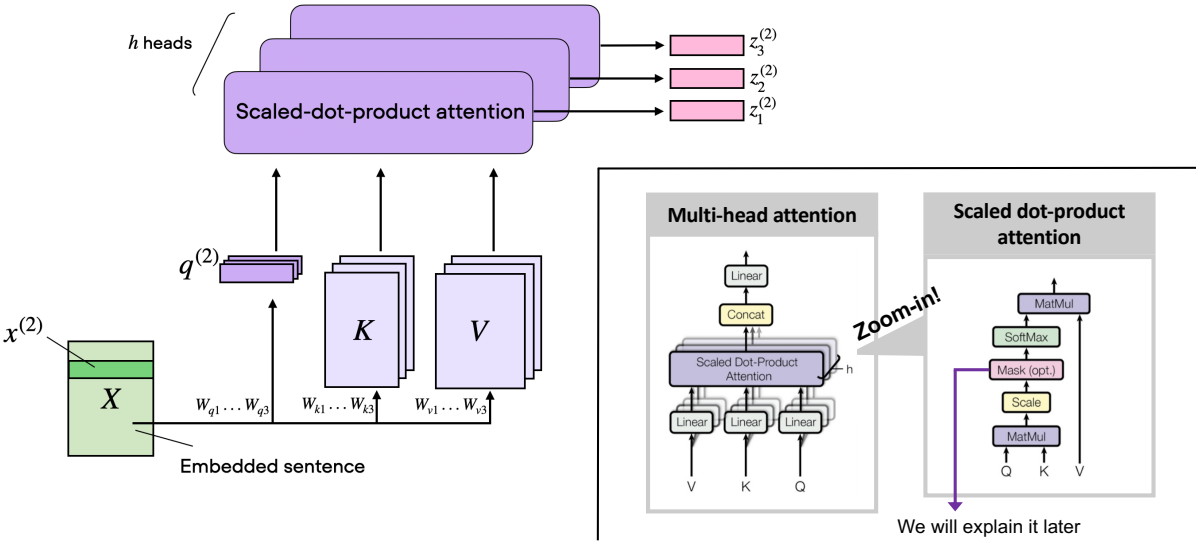
…

Multi-Head Attention: Same Sequence, More Alignments

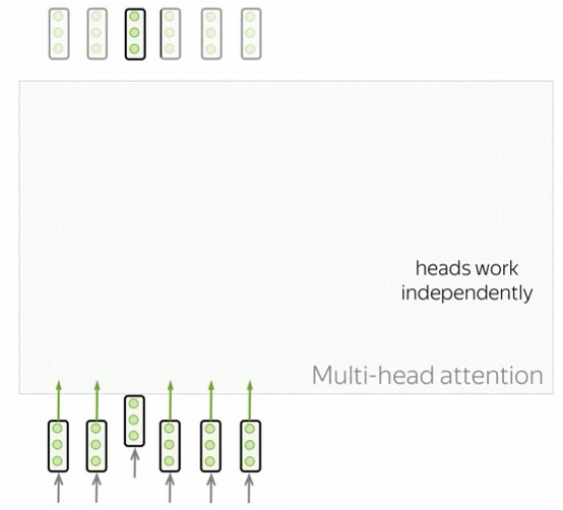
- Use a set of parallel self-attention layers, known as heads
  - Total of 8 in the original Transformer
- Each head has distinct learnable matrices
  - Allowing it to independently examine the entire input sequence
- Heads' output vectors are concatenated and mapped to a single vector through an additional learnable matrix



Multi-Head Attention: Two Popular Diagrams

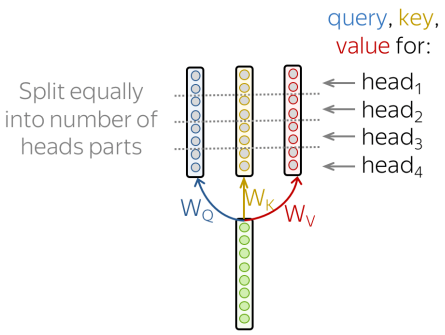


Multi-Head Attention: Inner Working Visualization



Multi-Head Attention: Size Trick Variant

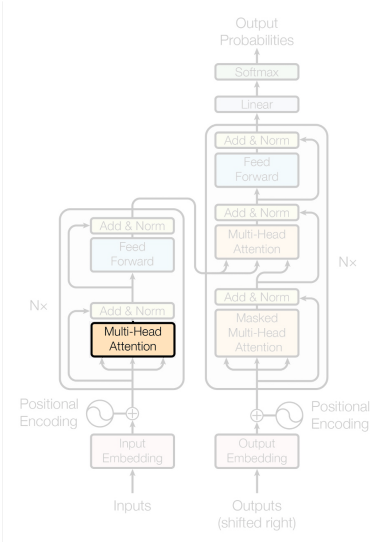
- In practice, we just split the queries, keys, and values we compute for a single-head attention into several parts
- In this way, models with one attention head or several of them have the same size (i.e., multi-head attention does not increase model size)



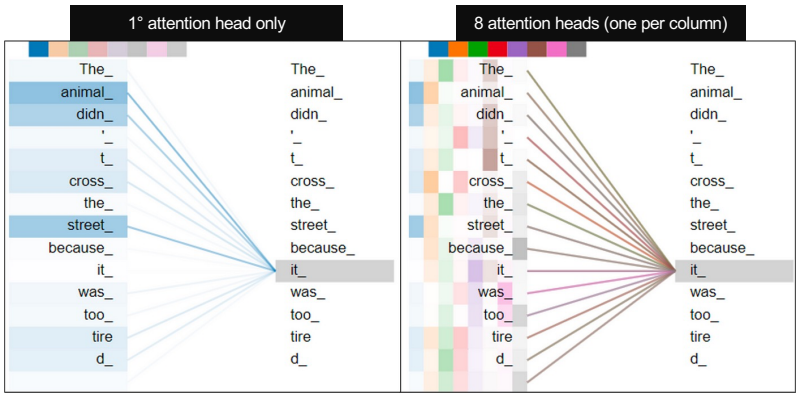
$$d_k = d_v = \frac{d_{model}}{h}$$

Where h = number of attention heads

Multi-Head Attention: Practical Example



- Attention heads learn multi-view syntactic and semantic relationships



Attention: the Off-By-One bug

- Multi-Head Attention is essentially encoding different facets of the input
  - What if one head's output logit is very small (approaching $-\infty$) and we want to ignore it ?
- The standard softmax operation prevents zero contribution from one head because of the normalization

$$softmax(X)_i = \frac{e^{x_i}}{\sum_{j=1}^{d} e^{x_j}}$$

$$\lim_{x_i \to -\infty} softmax(X) = \frac{1}{d}$$

L'Hôpital's Rule (!!)

Hidden state dimension

- A very simple but powerful trick to enable null-contributions is to add a balancing factor to the denominator

$$softmax_{\mathbf{1}}(X)_i = \frac{e^{x_i}}{\mathbf{1} + \sum_j e^{x_j}}$$

$$\lim_{x_i \to -\infty} softmax_{\mathbf{1}}(X) = 0$$

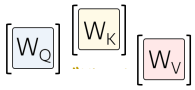
GPT-OSS (Aug '25) extends this trick by replacing "+1" with a learnable bias

Insights

Read more about Softmax optimization: "Attention is off by one", "Softmax is not Enough", "Gumbel-Softmax"

Rationale for Q, K, V Separation: Q≠K

- Three is *not* a magic number
  - The triple-matrix design is a deliberate solution for capturing relations within sequential data
  - Using two or four projection matrices would either limit the model's flexibility or introduce unnecessary complexity



Why not less than 3 projection matrices ?

- If we set Q = K the attention models only simmetric relation between token
  - setting Q=K forces the relationship to be symmetric because Wq = Wk = W

$$S = \frac{QK^\top}{\sqrt{d_k}} = \frac{XWW^\top X^\top}{\sqrt{d_k}} \qquad S_{ij} = S_{ji} \text{ for all } i, j.$$

The robot dropped the ball becuse **it** was heavy

This asymmetric relationship cannot be modeled

High attention. The pronoun needs the noun for its meaning.

Low attention. The pronoun adds no new context to the noun.

Rationale for Q, K, V Separation: V≠K

- **The necessity of a distinct value**
  - The role of V is to decouple the information used for determining relevance (via Q-K interaction) from the information that is actually aggregated
  - If K and V were the same, the aspects of a token that make it relevant would be the same aspects that are passed on

The **cat** **sat** on the mat

When processing the word **«sat»**…
- The **«sat»** token's **query** might be interested in finding the agent (the cat)
- The **«cat»** token's **key** would signal its role as the subject
- However, the information we want to extract from the **«cat»** token (its **value**) might be its semantic meaning

Deleting Q, K or V would cause the model to lose degrees of freedom.
It would need more capacity elsewhere to make up for the lost expressiveness.

Rationale for Q, K, V Separation: Research Attempts

- Some researchers have tried to use fewer projection matrices…
- Kowsher et al. (2025) proposed a variant called **Shared-Weight Self-Attention**
  - They share a single base matrix for Q, K, and V…

$$S = XW_S$$

  - … But still maintain *some distinctiveness* among them: to derive queries, keys, and values, they multiply S with three diagonal matrices

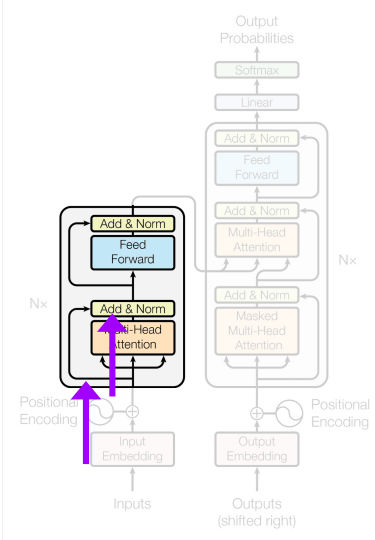
$$Q = SD_Q, K = SD_K, V = SD_V \qquad \text{So, } W_Q = W_S D_Q, W_K = W_S D_K, W_V = W_S D_V$$

  - This approach is an empirical trick for efficiency (66% reduction in attention-block parameters) that still allows competitive performance on GLUE tasks
  - Not evidence that the traditional triad scheme is *suboptimal*

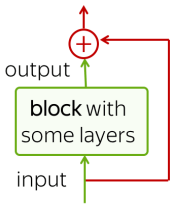
Rationale for Q, K, V Separation: No Need for Other Roles

- **If three matrices are good, wouldn't four be better ?**
  - A fourth matrix would necessitate a new, distinct role within the attention mechanism
  - However, it is not clear what fundamental aspect of conceptualization is missing
  - The existing triad elegantly covers the essential components of an IR system
  - A hypothetical fourth matrix would likely introduce redundancy or a level of complexity that does not translate to a significant improvement in performance
  - In essence, the three-role system of Q, K, V already strikes a balance between expressive power and architectural efficiency

Residual Connections: The «Add» Part

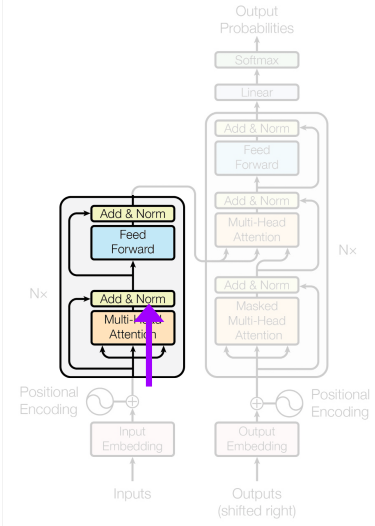


- After each step, the original information is reintroduced (add a block's input to its output)
- Help the model to retain the important details while still making progress in understanding the input
  - E.g., Positional Encoding
- They yield better learning due to stable gradients (healthy magnitude, neither too small nor too large)
  - They ease the gradient flow through a network and allow stacking a lot of layers



Residual connection: add a block's input to its output

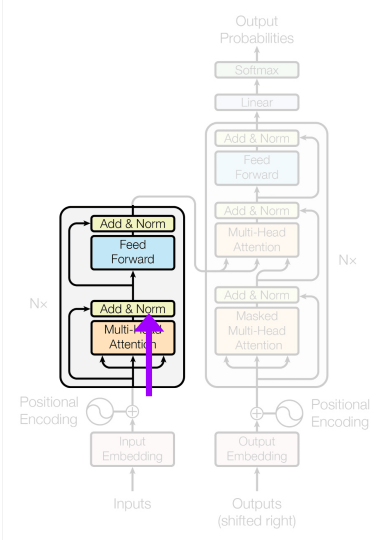
Normalization Layer: The «Norm» Part



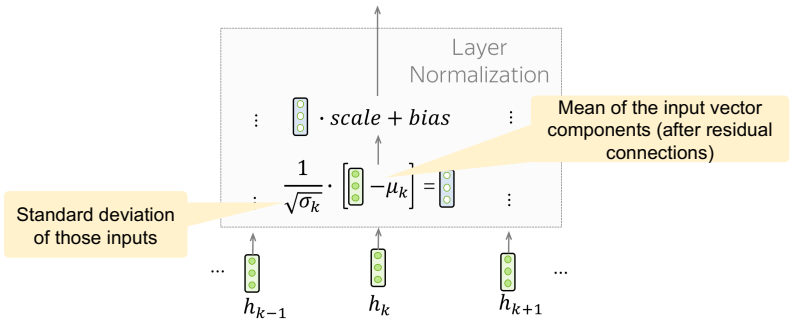
- It independently normalizes vector representation of each example in the batch
  - Control "flow" to the next layer
- It centers token embeddings around the origin
  - Ensure they have mean=0 and standard deviation=1
- It provides stability during training
  - Larger learning rates without causing the model to diverge
- It reduces "covariate shift" (type of overfitting issue)
  - It occurs when the model has learned to make predictions based on a certain data distribution during training, but that distribution no longer holds true during testing

Note; An embedding is a vector of floating-point numbers without boundary constraints. Only the Sinusoidal PE score, which is added to the embedding, falls within the normalized range [-1, 1]. This is, moreover, one of the reasons for the presence of normalization which help prevent activation values from becoming excessively high or low throughout the layers.

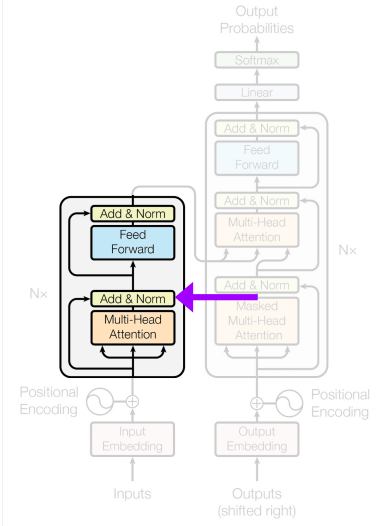
Normalization Layer: Trainable Parameters



- LayerNorm has trainable parameters: ***scale, bias***
  - They are used **after normalization** to rescale layer's outputs (or next layer's inputs)
  - They are **layer parameters** (i.e., do not change depending on the input hidden representations)



Add and Normalize Math



- It receives the input (X) and output (Y) matrices from the multi-head self-attention layer
- Suppose X and Y both have size $n \times d_{model}$

Mathematical overview, out: Z

- Adding: $A = X + Y$

Residual or skip connection

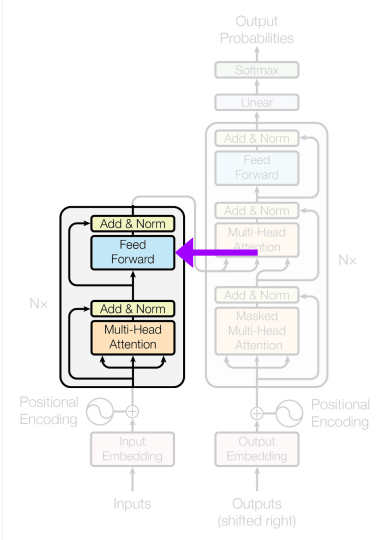
- Normalizing:

$$\mu = \frac{1}{n \times d_{model}} \sum_{i=1}^{n} \sum_{j=1}^{d_{model}} A_{ij} \qquad \sigma^2 = \frac{1}{n \times d_{model}} \sum_{i=1}^{n} \sum_{j=1}^{d_{model}} (A_{ij} - \mu)^2$$

$$Z_{ij} = \frac{A_{ij} - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

Standardizes each element using the mean and variance, with a small epsilon (ε) added for numerical stability

Feed Forward Layer: Introducing Non-Linearity

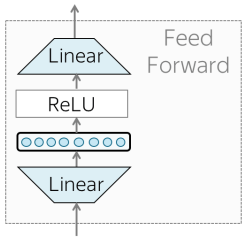


- Attention = **linear** transformations
- In addition to attention, each Transformer block has a feed-forward layer
  - Two linear layers with ReLU **non-linearity** between them

$$FF(x) = \max(0, xW_1 + b_1)\, W_2 + b_2$$

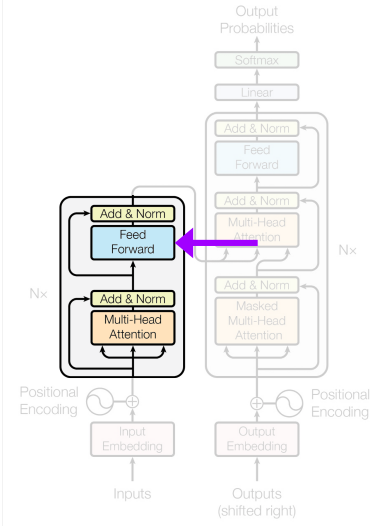
The result of several linear combinations (here 2) remains a linear combination

$$(xW_1 + b_1)W_2 + b_2 = xW' + b'$$



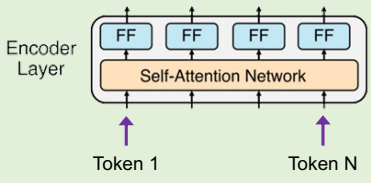
- **Attention:** "look at other tokens and gather information"
- **FF:** "take a moment to think and process this information"

Feed Forward Layer: Parallelization



- The feed-forward layer is applied to each token embedding, i.e., token-wise mapping
  - Each token representation can be processed independently of the others
  - Weights are shared among tokens ⇒ it limits the number of parameters and helps the network generalize better to sequences of varying length

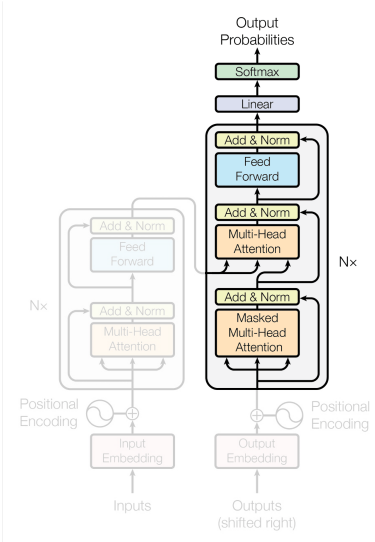
Efficiency



- Each token in each position flows through its own path in the encoder
- Cross-path dependencies are captured by the self-attention sub-layer

…

Decoder: Autoregressive Generation

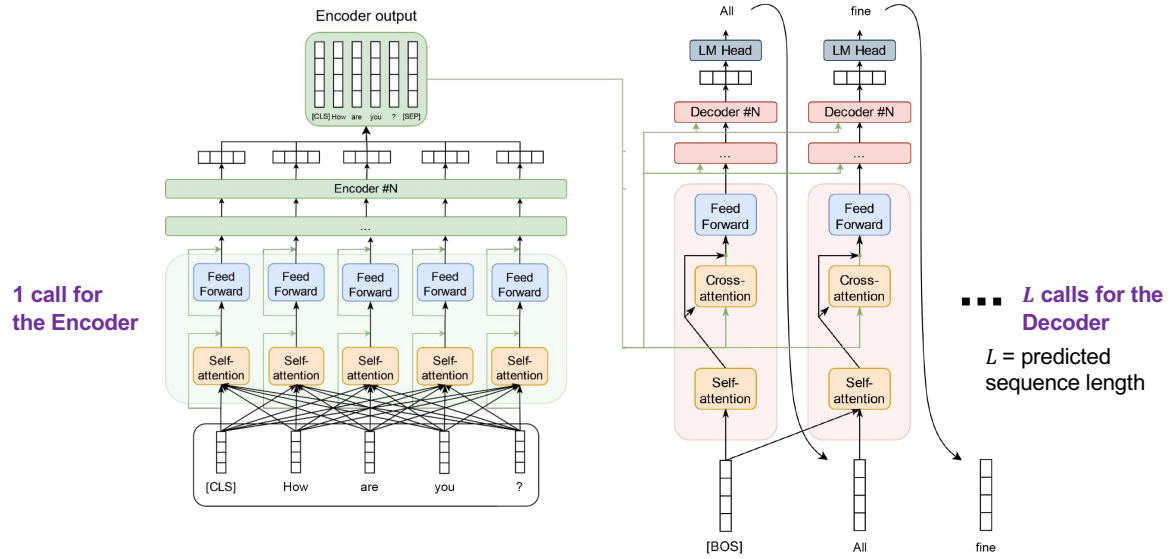


- The decoder generates the output sequence incrementally, token after token
  - Two inputs for each prediction step: **(i)** the Encoder's output, **(ii)** the previous output basic embedding

Inference

Input	Output
How are you?	BOS
	BOS All
	BOS All fine
	BOS All fine thanks

Multiple Decoder Calls: A Deeper Look At Inference

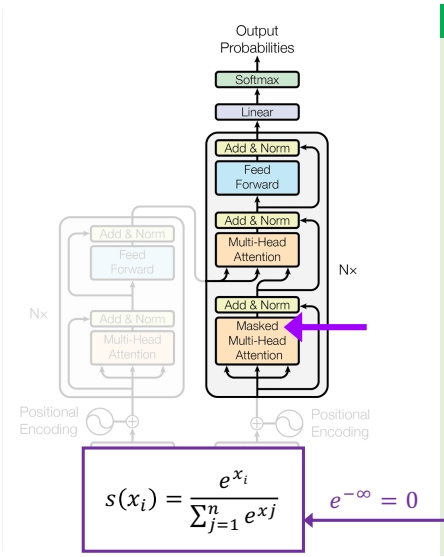


1 call for the Encoder

... L calls for the Decoder

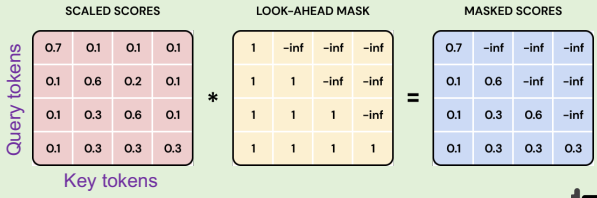
L = predicted sequence length

What About Training? Masked Self-Attention



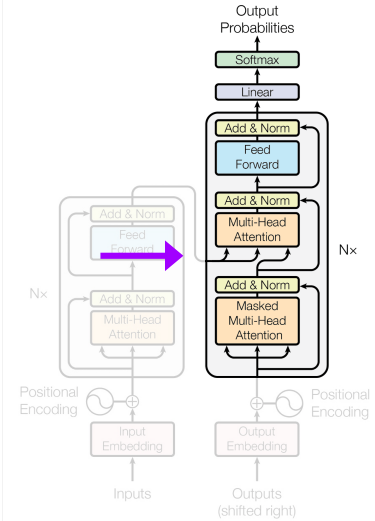
Efficiency

- To speed up training, **next token probabilities are computed in parallel**; i.e., the target sequence is fed into the decoder at once
- This is **only possible during training** since we obviously do not have access to the target sequence at inference time
- The network must not cheat: it cannot attend to future output tokens; therefore, a **constant triangular matrix (mask)** is added to QK^T
- All values **above the diagonal** are query tokens attending key tokens in the future (that must be **masked to ensure embeddings never depend on future tokens**)



After the attention softmax, all $-\infty$ values will become equal to 0

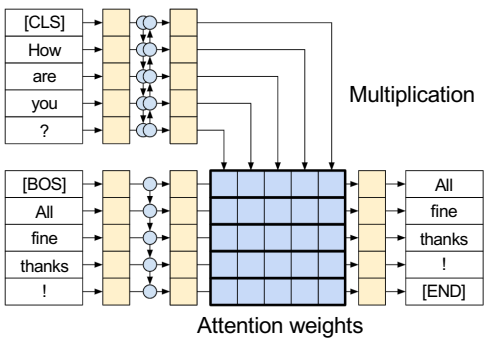
Decoder: Cross-Attention



- Inter-sequence attention: information from the previous output sequence (decoder) is aligned with information from the input sequence (encoder)
- It allows the model to attend to relevant parts of the input when generating each output token

K (target) and V (information to be retrieved) = Last-layer Encoder token embeddings (bidirectional attention)

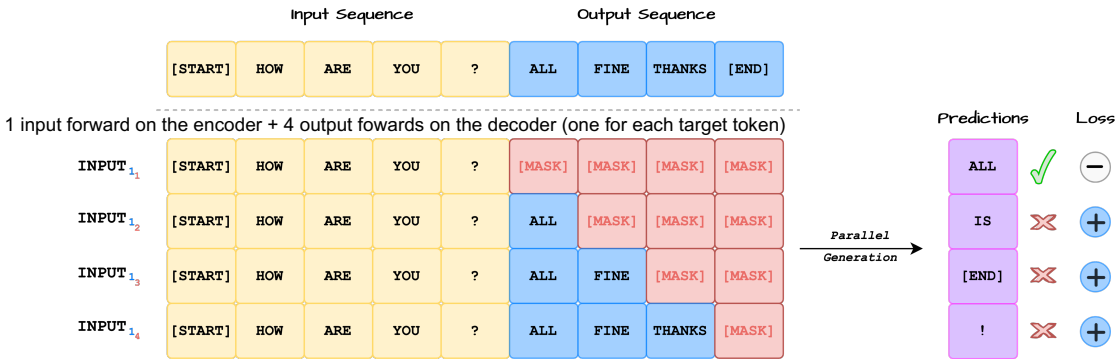
Q (source) = Decoder token embeddings (monodirectional attention on past tokens only)



Teacher Forcing: Parallelizable Token-Level Loss

During training, each output token is **predicted in parallel** by setting a sliding attention mask over the visible input tokens

A loss signal is computed for each generated token depending on its correctness w.r.t. the ground truth. **Eventual errors do not affect the other concurrent predictions.**

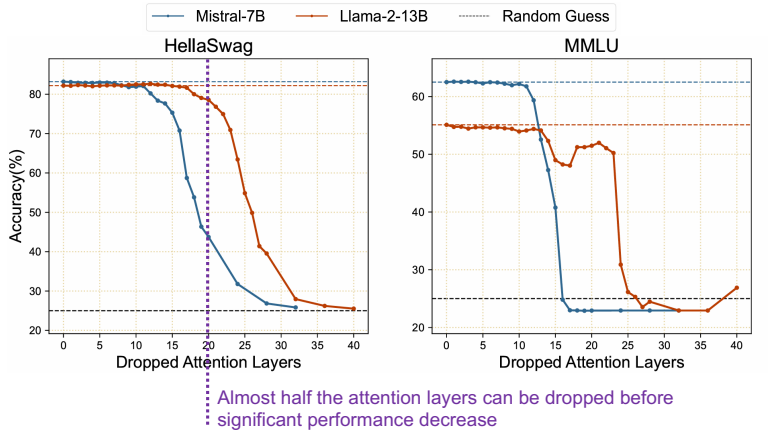


What Matters in Transformers?

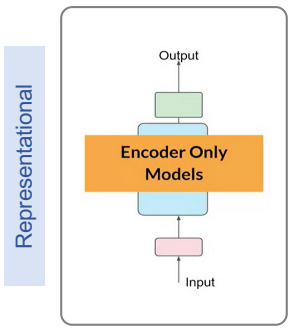
- **There might be lots of computational redundancy in larger architectures**
- He et al. (2024) tried to delete attention layers, FF layers, or entire blocks
  - Removing **entire transformer blocks** leads to significant performance degradation
  - Removing **FF layers** results in significant performance degradation (proving the importance of non-linear layers)
  - Removing **attention layers** causes almost no performance degradation 🚀
- Clarification: similarity-based drop without retraining
  - The attention layers were not removed randomly but according to a similarity-based metric
  - Hypothesis: redundant modules produce outputs that are similar to their inputs (minimal transformation); instead, important modules are expected to significantly alter their inputs
  - The layers are removed in a one-shot fashion and no (re)training is required after removal
  - However, retraining the model after the removal could potentially even recover some of the lost performance 💡

Attention Layers Removal in Large Language Models

- In LLaMA-2 70B, even if half of the attention layers are deleted (which results in a 48% speed-up), there is only a 2.4% decrease in model benchmarks
  - The authors recently added LLaMA-3 results to the paper, which are similar



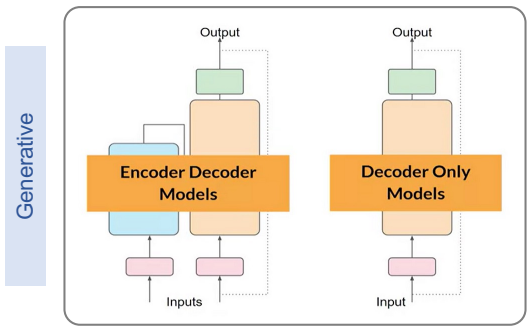
Architectural Variations



Encoder

Takes text (or other data) as input and outputs a dense representation of it

- **Example:** BERT
- **Use cases:** classification, information retrieval, information extraction
- **Typical size:** millions of parameters



Encoder-Decoder

The encoder first processes the input seq into a context representation, then the decoder generates an output seq

- **Example:** T5, BART
- **Use cases:** translation, summarization, paraphrasing
- **Typical size:** millions of parameters

Decoder

Generate new tokens to complete a sequence, one token at a time

- **Example:** GPT, LLaMA
- **Use cases:** text generation, chatbots, code generation
- **Typical size:** billions of parameters

Pre-Training and Fine-Tuning

- For every new text-based application, having to start from scratch by teaching the model the basics of language would be highly costly and inefficient

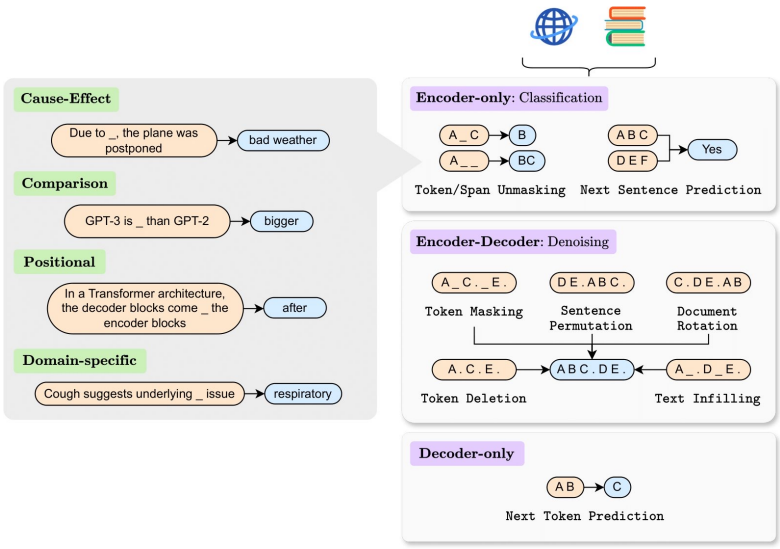
Pre-training

- **Data:** Large amounts of generic, raw text
- **Objective:** Predict the next token using automatically created input-output examples, without human annotation
- **Result:** The model acquires syntax, semantics, and general knowledge
- Generally performed only once

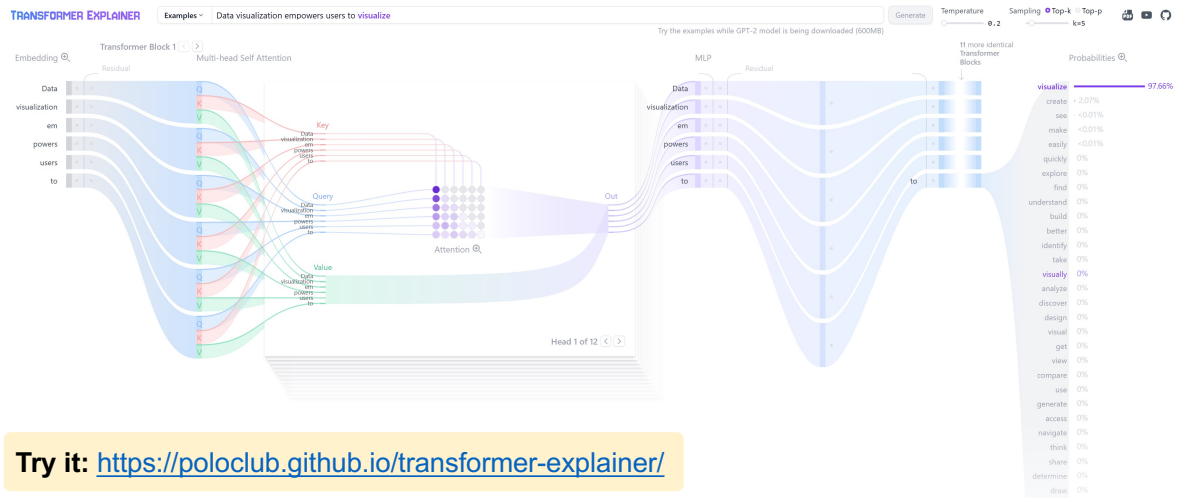
Fine-tuning

- **Data:** Specific examples for the task of interest (e.g., QA pairs, document-summary pairs)
- **Objective:** During fine-tuning, a generative LM continues to predict the next token, but in a context targeted for the specific task
- **Result:** The model specializes its knowledge
- Can be performed multiple times to create specialized copies for different tasks (or languages and domains)

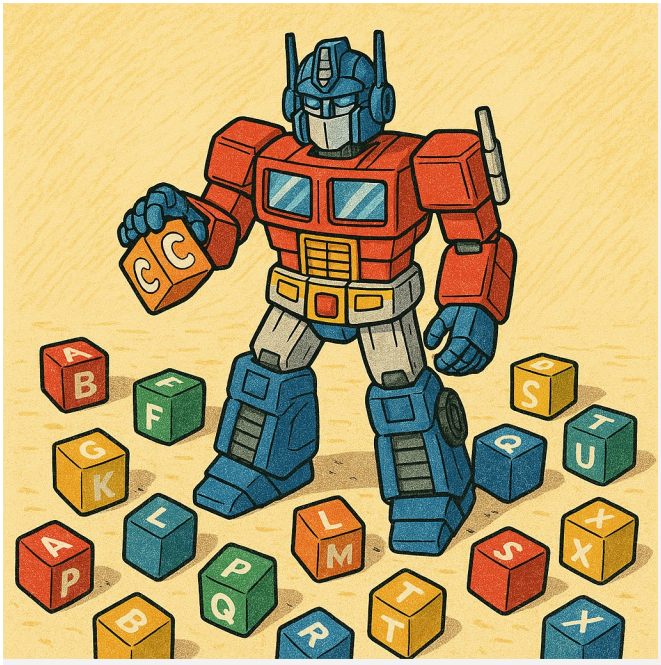
Self-Supervised Pre-Training



Interactive Visualization of GPT-2



Try it: https://poloclub.github.io/transformer-explainer/



Decoding Strategies

Inference (test) time configuration

Probabilistic Generation

- We consider autoregressive models, with trainable weights θ, that define the conditional probability $p_\theta(\boldsymbol{y}|\boldsymbol{x})$ of a target sequence $\boldsymbol{y}$ given a sequence $\boldsymbol{x}$
- The output space of all possible output sequences is $\mathcal{Y} = \{v \circ \text{EOS} \mid v \in V^*\}$
  - $\circ$ denotes string concatenation and V^* is the Kleene closure of V
  - Kleene closure = the set of all possible finite sequences that can be formed by concatenating elements from V
- The model factorizes the probability of $\boldsymbol{y}$ as follows:

$$p_\theta(\boldsymbol{y}|\boldsymbol{x}) = \prod_{t=1}^{|\boldsymbol{y}|} p_\theta(y_t|\boldsymbol{x}, \boldsymbol{y}_{<t})$$

Each $p_\theta(\cdot\,|\boldsymbol{x}, \boldsymbol{y}_{<t})$ is a distribution over $\bar{V} = V \cup \text{EOS}$; i.e., model output after softmax

Commonly, weights θ are learned by minimizing cross-entropy loss $\mathcal{L}(\theta, C)$ between predicted tokens and ground-truth ones in a training corpus C

Numerical stability (prevents underflow and keep calculations in a more manageable numerical range)

$$= \sum_{t=1}^{|\boldsymbol{y}|} \log p_\theta(y_t|\boldsymbol{x}, \boldsymbol{y}_{<t})$$

Need for Heuristics

- The purpose of decoding is to generate a hypothesis, $\boldsymbol{y}^*$
- Ideally, it should be defined as the one that maximizes the conditional probability $\boldsymbol{y}^* = \text{argmax}_{\boldsymbol{y} \in \boldsymbol{Y}} p_\theta(\boldsymbol{y}|\boldsymbol{x})$, i.e., the single highest-probability hypothesis
  - The number of generations in the symbolic space increases as $|V|^{|\boldsymbol{y}|}$ ...
  - ... So, the exact search (a.k.a. *maximum a posteriori, MAP*) is NP-hard
- Even if a solution were tractable, MAP would be far from high-quality text
  - MAP favors "safe" generic choices at each step – leading to bland or trivial replies
  - Shorter responses accumulate higher overall probability (length bias)
  - MAP is linguistically conservative, avoid any less-common word, and lacks the variance and richness of human writing
  - Repetitive output; i.e., the same high-probability words or phrases over and over
- Thus, decoding is approximated with heuristics (a.k.a. decoding strategies)

Decoding Strategies: Two Families

- A decoding strategy defines the procedure for selecting the next output token based on the model's predicted probability distribution
  - Component **outside the model** that actually turns distribution probabilities above V into text
- Decoding strategies can be categorized into two main classes

Deterministic	Stochastic
Optimize for continuations with high probabilities	Introduce elements of randomness into the generation process

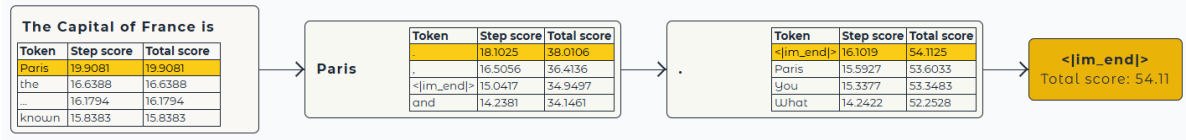
⚠️ Note that all strategies terminate when y_t reach EOS for some $t < \max_length$

Deterministic: Greedy Search

- The default decoding strategy on Hugging Face
- It selects the most probable token at each step (locally-optimal decision)
  - ✅ It works well for tasks with relatively short outputs where creativity is not a priority
  - ❌ It breaks down when generating longer sequences because it is prone to repeat itself

$$y_t = \underset{y \in \overline{V}}{\operatorname{argmax}} \log p_\theta(y | \boldsymbol{x}, \boldsymbol{y_{<t}}) \quad (\text{for } t > 0)$$

Real example from SmolLM2-360M-Instruct (decoder-only). ***Play with it!***



Deterministic: Beam Search

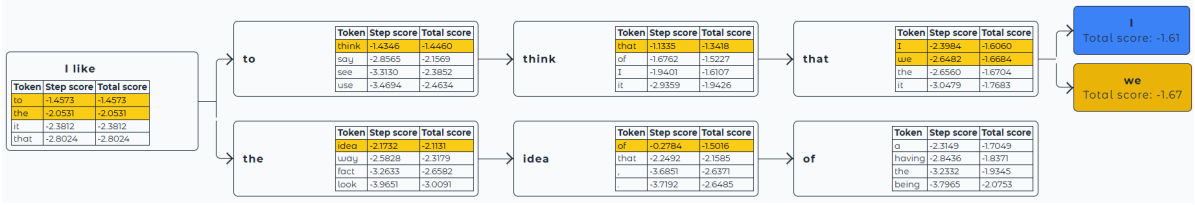
- It selects the next token based on how good the *entire resulting sequence* could be – not just the next immediate probability
  - A pruned breadth-first search algorithm that expands the hypothesis in a greedy left-to-right way, retaining the top-b candidates at each t

$$Y_t = \underset{\substack{Y' \subseteq \mathcal{B}, \\ |Y'|=b}}{\operatorname{argmax}} \mathcal{S}(Y'_t) \quad (\text{for } t > 0)$$

$\mathcal{S}: \boldsymbol{\mathcal{Y}} \to \mathbb{R}$ is a scoring function on $\mathrm{Y} \subseteq \boldsymbol{\mathcal{Y}}$. By default, $\mathcal{S}(Y) = \sum_{\boldsymbol{y} \in Y} \log p_\theta(\boldsymbol{y}|\boldsymbol{x})$ but it can be extended w/ penalties

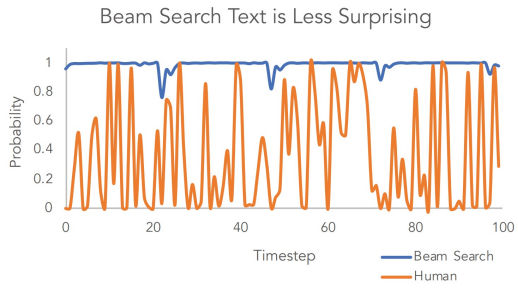
$\mathcal{B}$ denotes all possible roll-out extensions $\{\boldsymbol{y}_{t-1} \circ y \mid y \in \bar{V} \text{ and } \boldsymbol{y}_{t-1} \in Y_{t-1}\}$

Real example from GPT-2 (decoder-only). ***Play with it!***



When Randomness is Needed

- In open-ended text generation or conversational tasks, less-predictable, creative texts from stochastic strategies are generally preferred
  - High-quality human language does not follow a distribution of high probability next tokens [Holtzman et al. 2019]
  - In other words, as humans, we want generated text to surpise us and not be boring



Beam Search

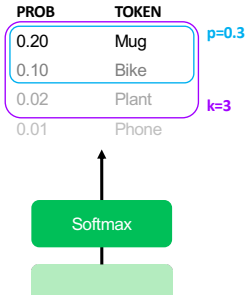
...to provide an overview of the current state-of-the-art in the field of computer vision and machine learning, and to provide an overview of the current state-of-the-art in the field of computer vision and machine learning, and to provide an overview of the current state-of-the-art in the field of computer vision and machine learning, and to provide an overview of the current state-of-the-art in the field of computer vision and machine learning, and...

Human

...which grant increased life span and three years warranty. The Antec HCG series consists of five models with capacities spanning from 400W to 900W. Here we should note that we have already tested the HCG-620 in a previous review and were quite satisfied With its performance. In today's review we will rigorously test the Antec HCG-520, which as its model number implies, has 520W capacity and contrary to Antec's strong beliefs in multi-rail PSUs is equipped...

Stochastic: Sampling (Pure, Top-k, Top-p)

- Output token at random using the probability distribution to weigh the selection
  - **Top-k**: the next token is drawn from the k tokens with the highest probabilities
  - **Top-p (nucleus)**: the next token is drawn from the smallest possible set of tokens whose cumulative probability exceeds p; this way, the pool size can dynamically increase/decrease

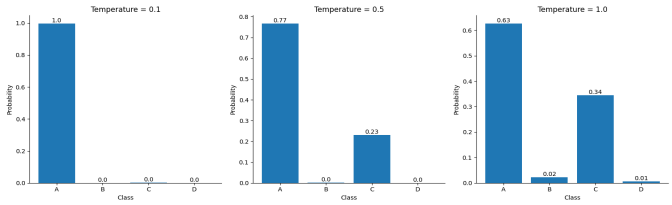


Temperature T: Influence the shape of the probability distribution through a scaling factor in the softmax layer

$$s(x_i) = \frac{e^{x_i/T}}{\sum_{j=1}^{n} e^{x_j/T}}$$

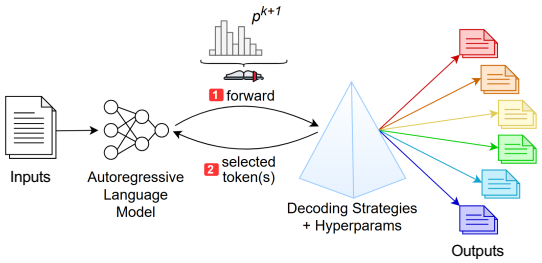
$T < 1$ Peak distribution — Less creative

$T > 1$ Broader distribution — More creative



Do Not Underestimate the Decoding Strategy Impact

- As tokenization, one of the most overlooked aspects of a language model
  - With the same frozen model, a decoding strategy can drastically change the generated text
- We assessed the impact of decoding strategies on abstractive summarization
  - 3,500+ inference runs with 9 decoding strategies (extensive hyperparameter sweep), 5 base models (both encoder-decoder and decoder-only), 6 datasets, 10 automatic metrics
  - The largest systematic study in literature

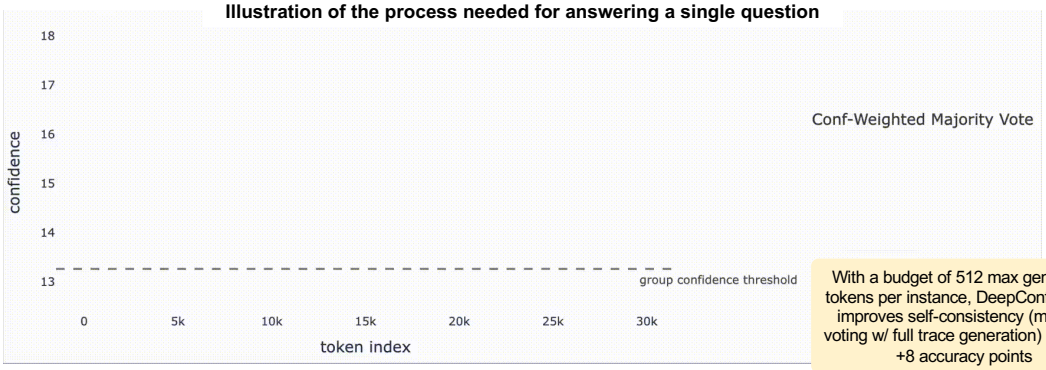


Not just about decimals. The decoding strategy choice con result in variations of up to 20 ROUGE and 9 BARTScore points.

There is no one-size-fits-all strategy. Tuning would be required.

Filter Generations with Confidence

- DeepConf [Fu et al., 2025] decides the model answer based on its confidence
  - It runs parallel thinking (multiple traces sampled w/ top-k and top-p), keep only the most confident ones (top-η%), and select the answer by confidence-weighted majority voting
  - Also appliable *online*, i.e., halting trace generation if confidence falls below a threshold, determined offline as the $(100-\eta)^{th}$ percentile of sample confidences



With a budget of 512 max generated tokens per instance, DeepConf@η=10 improves self-consistency (majority voting w/ full trace generation) by up to +8 accuracy points

ALMA MATER STUDIORUM UNIVERSITY OF BOLOGNA
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING – DISI

Large Language Models: Prompting and Fine-tuning

Prof. Gianluca Moro
Tutors: Dr. Giacomo Frisoni, Dr. Lorenzo Molfetta

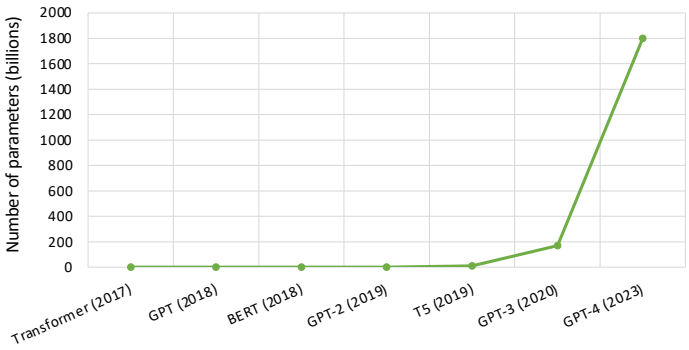
DISI, University of Bologna, Cesena
Via dell'Università, 50 I-47522 Cesena (FC), Italy
{name.surname}@unibo.it

Outline

- **Overview**
- Instruction Fine-tuning
- Prompting

Architectural Growth Trends

- Many studies have shown that increasing the number of parameters within language models leads to performance improvements in various tasks
  - This has motivated training of increasingly large models (**exponential growth**)
  - In just 3 years, we have gone from 95M parameters to 1.2T

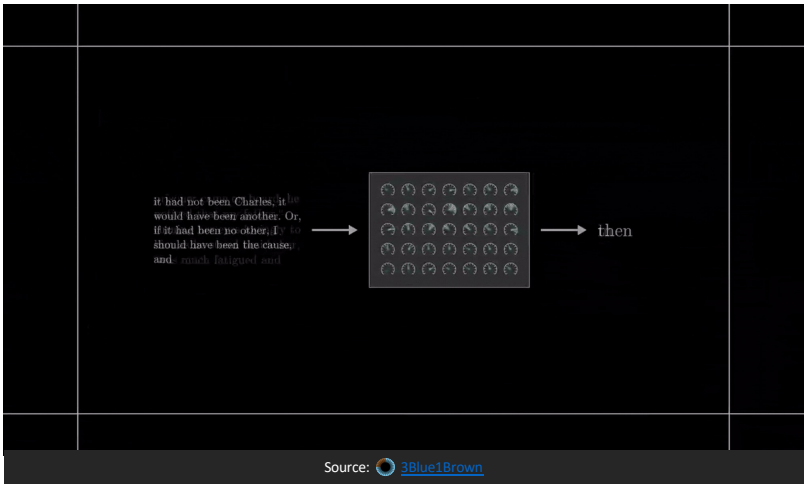


Moore's Law in NLP
Every year, the size of the models grows by a factor of
10×

Biological Inspiration
As in the human and animal brain, there is a positive correlation between brain size and cognitive ability

Large Language Model (LLM)

A Scale of Computation that is Difficult to Imagine



Source: 3Blue1Brown

Imagine being able to perform one billion additions and multiplications per second. Carrying out all the operations involved in training a modern LLM would take you well over **100 million years**.

This computational capability is made possible only through the use of specialized chips, optimized to perform many operations in parallel: **GPUs**

The Numbers of GPT-4

GPT4 Model Estimates

Training Size	Compute Size	Model Size
# of Book shelves for 13T tokens	Compute time for 2.15 e25 FLOPs	Size of Excel Sheet for 1.8T params
650 kms	7 million years	30,000
Long line of Library Shelves	On mid-size Laptop (100GFLOPs)	Football Fields sized Excel Sheet
100000 tokens per Book 100 Books per shelf 2 Shelves per meter	100GLOPs per second	1x1 cm per Excel cell 100 x 60 meters Field Size

Source: https://the-decoder.com/gpt-4-architecture-datasets-costs-and-more-leaked

How Much and What Data is Used for Pre-training ?

Model	# Token
Qwen3 (Jun 2025)	36T
Phi-3.5-MoE (May 2025)	4.9T
Qwen2.5 (Apr 2025)	18T
Phi-4 (Dec 2024)	9.8T
DeepSeek-V3 (Dec 2024)	15T
LLaMA-3 (Apr 2024)	15T
LLaMA-2 (Jul 2023)	2T
GPT-4 (Mar 2023)	13T
LLaMA (Feb 2023)	1.4T
GPT-3 (May 2020)	400B

- Main data categories
  - Public web (billions of web pages)
  - Digitized literature (e.g., books, scientific publications)
  - Encyclopedic resources (e.g., Wikipedia)
  - Source code (e.g., GitHub repositories with permissive license)
- Linguistic and thematic diversity
  - Over **110 languages** in the most used datasets, from general content to technical-scientific domains

It would take an average person over **2,600 years**, without ever resting, to read all the text used to train **GPT-3**

Quality > Quantity ?
In 2023, Meta revealed it had invested more resources in preparing the data for LLaMA-2 ($11M) than in the training process itself ($8M).

💡 **Foundation model:** a large-scale model trained on broad, diverse data such that it can serve as a general-purpose base adaptable to a wide variety of downstream tasks. It provides shared representations and transferable capabilities across domains.

Popular Pre-training Datasets

- Filtered web archives based on Common Crawl dumps
  - **C4 (1.4T T5 tokens)**, 750GB compressed web-extracted text from April 2019
  - **WebText2 (19B GPT-3 tokens)**, high-quality webpages voted throughout Reddit
  - **RefinedWeb (7T Falcon tokens)**
  - **Oscar (1.08B BLOOM tokens)**, 151 different languages
  - **CulturaX (6.3T SentencePiece tokens)**, 167 languages
- Books
  - **Project Gutenberg (215B Falcon tokens)**
- Conversations
  - **Reddit, HackerNews, StackOverflow, IRC, Youtube Subtitles (170B Falcon tokens)**
- Technical
  - **ArXiv, PubMed, USPTO, Wikipedia (60B Falcon tokens)**
  - **The Pile (340B Falcon tokens)**, pre-aggregated dataset
  - **GitHub (1T Falcon tokens)**, **The Stack (430B GPT-NeoX tokens)**

NLP Market: Huge Investments

Hugging Face
235M$ in 2023
4.5B$ evaluation

cohere
270M$ in 2023
2.2B$ evaluation

character.ai
150M$ in 2023
2.2B$ evaluation

Inflection
235M$ in 2023
1.3B$ evaluation

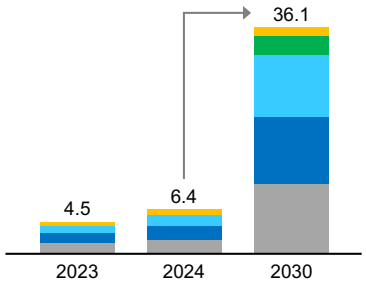
ANTHROP\C
550M$ in 2023
20-30B$ evaluation

OpenAI
10B$ in 2023
80-90B$ evaluation

Large Language Model Market
Global Forecast to 2030 (USD BN)

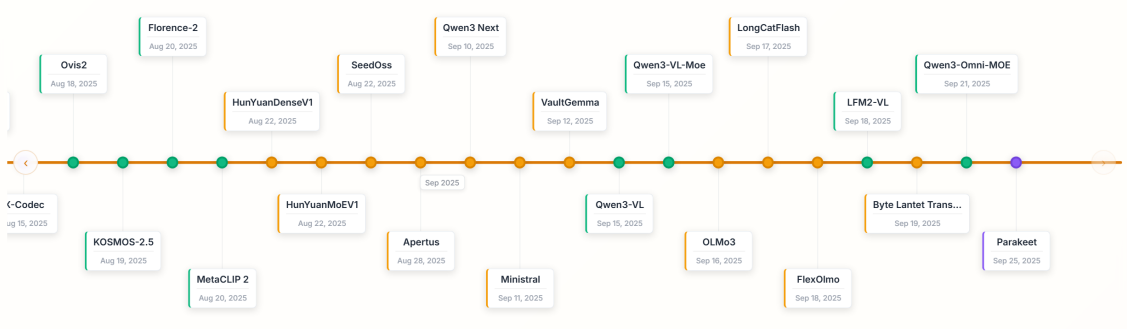
The global LLM market is expected to be worth USD 36.1 billion by 2030, growing at a CAGR of 33.2% during the forecast period [Source]

North-America Europe Asia-Pacific
Middle East and Africa Latin America



Models Timeline

- In September 2025, we passed 2M+ public models on HuggingFace

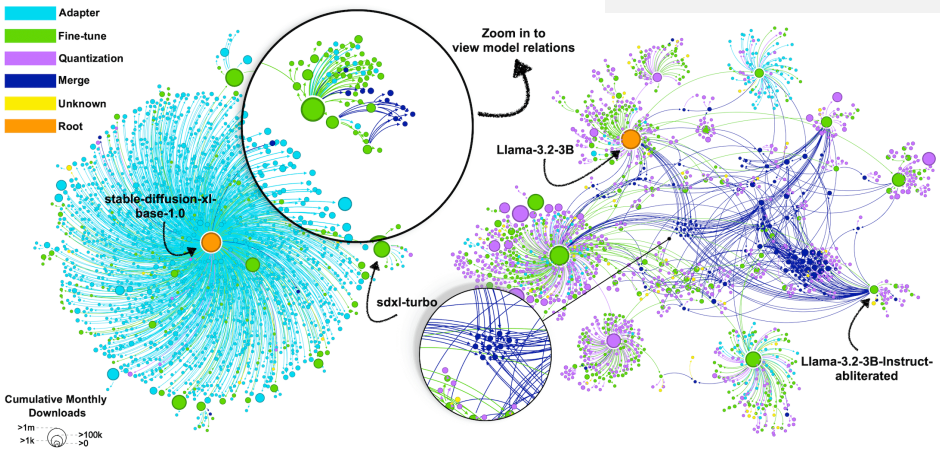


This HF Space can assist you in finding a suitable Transformer-based model for your task:
https://huggingface.co/spaces/yonigozlan/Transformers-Timeline

Models Atlas

Node = HF model, **Node size** = cumulative monthly downloads, **Edge** = weight transformations (e.g., fine-tuning), **Node distance** = distance between model weights, **Color** = transformation type

Horwitz et al., NeurIPS 2025 [Source]



Top 30% most download models in the Stable Diffusion and LLaMA regions.
The LLaMA region has a more complex structure and a wider diversity of transformation techniques.

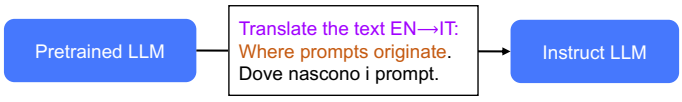
Outline

- Overview
- **Instruction Fine-tuning** ☞
- Prompting

Instruction Fine-tuning (IFT)

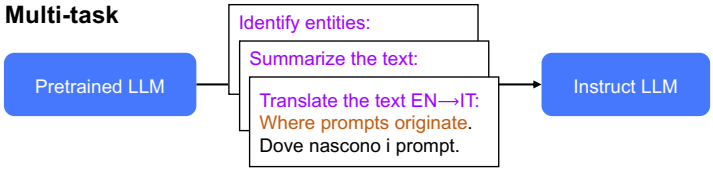
- Post-training adaptation method where a pretrained language model is optimized to follow natural language instructions (1 model, N tasks)
  - Trains on curated [instruction,input]-completion pairs ← IFT ⊂ Supervised Fine-tuning (SFT)

Task-specific



⚠ It can reduce the performance on other tasks (*catastrophic forgetting*). Not an issue if only a single task matters.

Multi-task



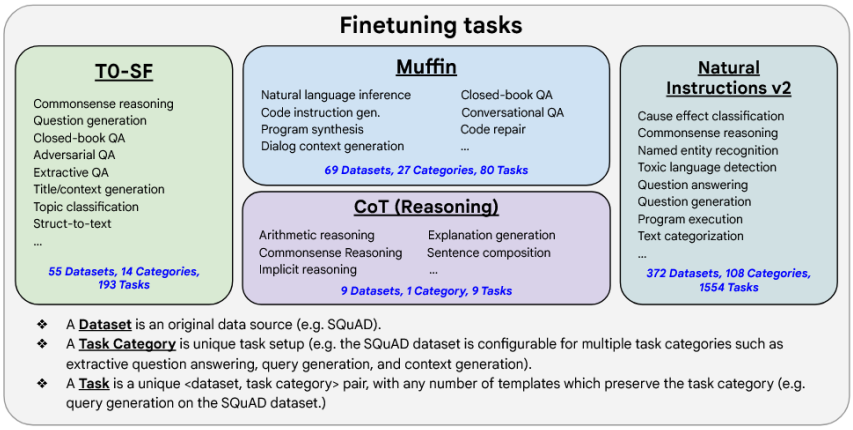
⚠ It requires a lot of data ($\sim 50 - 100K$ examples)

Model variants differ based on the dataset and tasks used during fine-tuning

The idea of casting all NLP tasks as text-to-text translations was conceptualized by Google's T5 [Raffel et al., 2019]

IFT: Example of Datasets from Flan-T5

- Flan-T5 [Chung et al., 2022] = IFT version of T5
  - Fine-tuned on 473 datasets, 1,836 tasks belonging to 146 task categories



⚠ IFT datasets frequently include synthetic examples

Automatic adaptation of existing datasets

Human annotation
E.g., for training InstructGPT, OpenAI sampled user inputs from GPT-3 API submissions and recruited 40 labelers to write desired responses

Self-instruct
Use an LLM to generate a series of possible instructions per task category and their corresponding responses

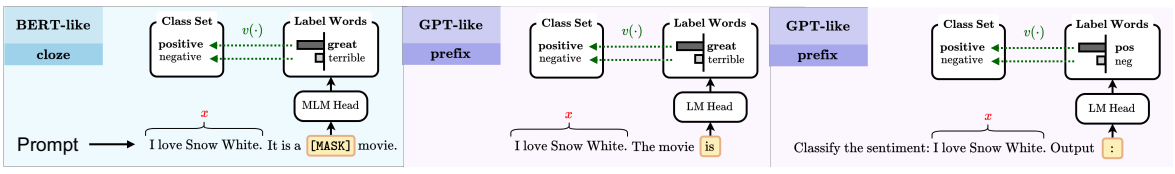
Outline

- Overview
- Instruction Fine-tuning
- **Prompting**

Prompting: An AI Paradigm Change

- Prompting is *how we talk* to an IFT (or pretrained) model to elicit a desired task behavior at inference time, leveraging its extensive parametric knowledge
  - *Goal?* Generalize to unseen tasks w/o fine-tuning 🔮
  - *How?* Reformatting inputs to mirror those encountered during training
- **Prompt template:** structured form used to guide the model in predicting masked or missing elements, depending on the goal it has been trained on
  - Formally, a function $f_\theta(\cdot)$ that maps an input x into a prompt x'
  - It can incorporate constant tokens (e.g., instructions) and input/output slots

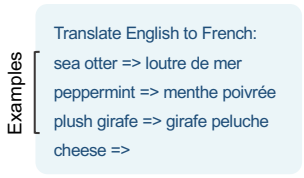
Example of prompting for sentiment classification with models trained with different objectives



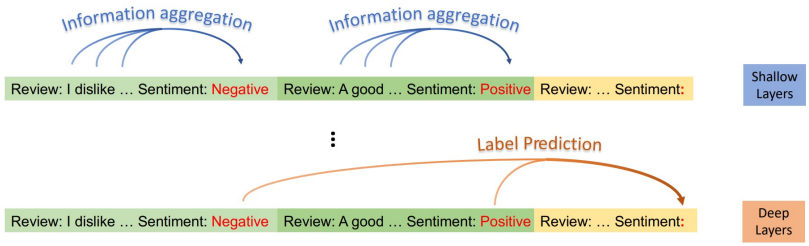
Pretrained on token unmasking | Pretrained on next token prediction | Pretrained + Instruct-tuned

In-Context Learning

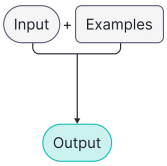
- Brown et al. (2020) reported that giving a few input-output examples as part of the prompt can provide clear performance boost across 42 benchmarks
  - Provide examples better inform the LLM about what it is expected to do



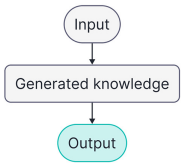
Information flow perspective from Wang et al. (2023), EMNLP 2023 Best Paper Award
In shallow layers, label words gather information from demonstrations to semantic representations for deeper processing, while deep layers extract and utilize this information from label words to formulate the final prediction



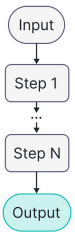
Prompt Tricks and Test-Time Scaling Techniques



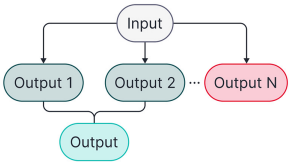
Few-shot
Input-output examples as part of the prompt



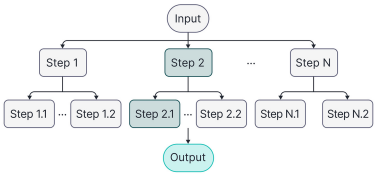
Generated context
Ask to first review what the LLM knows about the domain and then use this knowledge for generating the output



Chain-of-thoughts (CoT)
Encourage LLMs to think step by step before jumping to a conclusion



Self-consistency
Sample multiple responses and use majority votes to decide on the final response



Tree-of-thoughts
Extends CoT by retaining the results from multiple intermediate steps until the best overall chained solution is found

Since some of these techniques are orthogonal to each other, they can be combined for better performance

Manual Prompt Engineering

- IFT (or pretrained) models are known to be highly sensitive to minor prompt variations; a small perturbation can largely affect model performance
- Prompt engineering = the process of optimizing the tokens used in the prompt to achieve desirable predictions
  - It demands expertise and involves time-consuming trial-and-error iterations
  - The most effective prompts may not align with human intuition, as language models respond to statistical patterns rather than logical coherence
  - E.g., out-of-context prompting (a technique that introduces random, unrelated information into prompts) can improve robustness [Cotta and Maddison, 2024]

👨‍💻 **AI Prompt Engineer: one of the hottest tech jobs in 2023**, commanding salaries of up to $335,000 [Forbes]

After conducting extensive experiments, researchers have shared **best practices** [OpenAI's Prompt Engineering Guide], even opening **prompt marketplaces** (e.g., PromptBase)

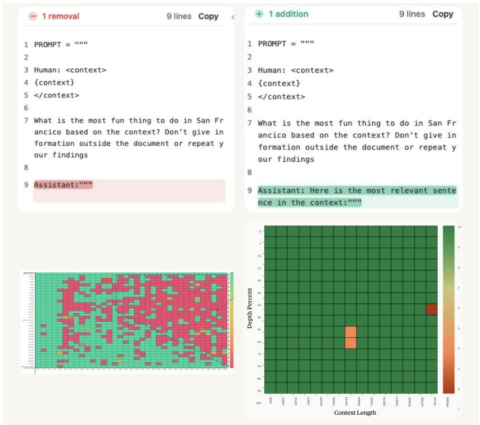
The Unreasonable Effect of Prompts

👨‍⚕️ Prompting allowed GPT-4 to gain up to +9 accuracy point on MedQA (Multiple-Choice Medical Question Answering on United States Medical License Exams), surpassing 90% - a threshold never reached before with ad-hoc fine-tuning [Nori et al., 2023]

Up to +50% in LLM math ability with *"Take a deep breath and work on solving this problem step by step"* [Yang et al., 2023]

Baselines				
PaLM 2-L	(Kojima et al., 2022)	A_begin	Let's think step by step.	71.8
PaLM 2-L	(Zhou et al., 2022b)	A_begin	Let's work this out in a step by step way to be sure we have the right answer.	58.8
PaLM 2-L		A_begin	Let's solve the problem.	60.8
PaLM 2-L		A_begin	(empty string)	34.0
text-bison	(Kojima et al., 2022)	Q_begin	Let's think step by step.	64.4
text-bison	(Zhou et al., 2022b)	Q_begin	Let's work this out in a step by step way to be sure we have the right answer.	65.6
text-bison		Q_begin	Let's solve the problem.	59.1
text-bison		Q_begin	(empty string)	56.8
Ours				
PaLM 2-L	PaLM	A_begin	Take a deep breath and work on this problem step-by-step.	**80.2**

From 27% accuracy score to 98% by adding *"Here is the most relevant sentence in the context"* [Anthropic's Claude Blog Post]



Thanks for the attention.

Is all you need.